

BRAVE-Based Real-Time Voice-to-Instrument Resynthesis

TREBALL FI DE GRAU DE

Jofre Geli de Fuenmayor

Director: Lonce Wyse & Xavier Lizarraga

Grau en Enginyeria de Sistemes Audiovisuals

Curs 2025–2026



**Universitat
Pompeu Fabra**
Barcelona

Escola
d'Enginyeria

Acknowledgements

The author would like to thank the project directors, Lonce Wyse & Xavier Lizarraga, for guidance throughout the iterative development and for the methodological flagging of the dataset heterogeneity concern that shaped Decision 3 (Appendix-D). Acknowledgement is also due to the ACIDS-IRCAM research group for releasing the RAVE and BRAVE architectures, training code, and pretrained checkpoints used as the project’s external baseline, and to the maintainers of the GuitarSet, Guitar-TECHS, IDMT-SMT-Guitar and Groove MIDI Dataset corpora.

Abstract

This thesis investigates whether the streaming-causal BRAVE architecture can deliver voice-driven instrument resynthesis at sub-10 ms latency when trained on heterogeneous, limited-scale public datasets. Six guitar and two drum training iterations were systematically evaluated under a custom diagnostic framework. The principal outcome is a guitar autoencoder that partially reconstructs in-distribution guitar audio — preserving harmonic structure and onsets, though with limited high-frequency detail — but does not generalise to voice: vocal input lies outside the guitar training distribution and was not rendered as recognisable guitar by any iteration. Reusing the same configuration template for a drums model did not transfer cleanly. Comparison against the official ACIDS-IRCAM percussion baseline showed it more responsive than the project’s drums model by more than two orders of magnitude on the same diagnostic, locating the limitation in training recipe and data scale rather than in the BRAVE architecture. The principal contributions are a five-mode failure taxonomy for RAVE-family models on small corpora, a cross-checkpoint calibration of the STFT-correlation diagnostic that exposes a previously unreported limitation of that metric, and an external-baseline disentanglement of architectural from training-recipe failure.

Resum

Aquesta tesi investiga si l'arquitectura BRAVE de tipus causal-streaming pot oferir resíntesi d'instruments controlada per veu amb una latència inferior a 10 ms entrenada sobre conjunts de dades públics, heterogenis i de mida limitada. S'han avaluat sis iteracions de guitarra i dues de bateria sota un marc diagnòstic propi. El resultat principal és un autoencoder de guitarra que reconstrueix parcialment àudio de guitarra en distribució — preservant l'estructura harmònica i els atacs, tot i que amb un detall limitat a altes freqüències — però no generalitza a la veu: l'entrada vocal queda fora de la distribució d'entrenament i cap iteració no la va reproduir com a guitarra reconeixible. La reutilització de la mateixa plantilla de configuració per a un model de bateria no es va transferir netament. La comparació amb la línia base oficial ACIDS-IRCAM de percussió la va mostrar més responsiva que el model de bateria del projecte per més de dos ordres de magnitud sobre el mateix diagnòstic, situant la limitació en la recepta d'entrenament i no en l'arquitectura BRAVE. Les contribucions principals són una taxonomia de cinc modes de fallada per a models de la família RAVE en corpus petits, una calibració entre punts de control de la mètrica de correlació espectral que exposa una limitació no documentada prèviament d'aquesta mètrica, i una desambiguació mitjançant línia base externa.

Resumen

Esta tesis investiga si la arquitectura BRAVE de tipo causal-streaming puede ofrecer resíntesis de instrumentos controlada por voz con una latencia inferior a 10 ms cuando se entrena sobre conjuntos de datos públicos, heterogéneos y de tamaño limitado. Se evaluaron sistemáticamente seis iteraciones de guitarra y dos de batería bajo un marco diagnóstico propio. El resultado principal es un autoencoder de guitarra que reconstruye parcialmente audio de guitarra en distribución — preservando la estructura armónica y los ataques, aunque con un detalle limitado en altas frecuencias — pero no generaliza a la voz: la entrada vocal queda fuera de la distribución de entrenamiento y ninguna iteración la reprodujo como guitarra reconocible. La reutilización de la misma plantilla de configuración para un modelo de batería no se transfirió limpiamente. La comparación con la línea base oficial ACIDS-IRCAM de percusión la mostró más responsiva que el modelo de batería del proyecto por más de dos órdenes de magnitud sobre el mismo diagnóstico, situando la limitación en la receta de entrenamiento y no en la arquitectura BRAVE. Las contribuciones principales son una taxonomía de cinco modos de fallo para modelos de la familia RAVE en corpus pequeños, una calibración entre puntos de control de la métrica de correlación espectral que expone una limitación no documentada previamente de dicha métrica, y una desambiguación mediante línea base externa.

Contents

Acknowledgements	i
Abstract	ii
Resum	iii
Resumen	iv
1 Introduction	1
1.1 Motivation	1
1.2 Research Questions	1
1.3 Objectives and Scope	2
1.4 Methodology	2
1.5 Original Contributions	3
1.6 Document Structure	3
2 State of the Art	4
2.1 Neural Audio Synthesis Architectures	4
2.2 Timbre Transfer and Voice Conversion	7
2.3 Neural Audio Synthesis Datasets	8
2.4 Research Gap and Motivation	9
3 Methodology	10
3.1 System Architecture	10
3.2 Training Paradigm	11
3.3 Datasets and Preprocessing	13
3.4 Diagnostic Methodology	14
3.5 Methodological Assumptions and Limitations	16
4 Implementation	18
4.1 Environment and Compatibility	18
4.2 Training Pipeline	18
4.3 Real-Time Deployment	19
4.4 Evaluation Tooling	20
5 Results and Evaluation	21

5.1	Guitar: Six-Iteration Arc	21
5.1.1	Iterations v1–v2: Latent Space Failures	21
5.1.2	Iterations v3–v5: Adversarial Instabilities	22
5.1.3	Iteration v6: Partial In-Distribution Reconstruction	23
5.1.4	Calibration of the STFT-Magnitude-Correlation Diagnostic	24
5.2	Drums	25
5.3	External Baseline: IRCAM Percussion	26
5.3.1	Candidate Explanations for the IRCAM Responsiveness Gap	26
5.4	Failure-Mode Taxonomy	28
5.5	Noise-Floor Heterogeneity in the Guitar Corpus	28
6	Conclusions and Future Work	30
6.1	Summary of Findings	30
6.2	Contributions	30
6.3	Limitations	31
6.4	Future Work	31
6.5	Closing Reflection	32
	References	33
	Appendix A Demo Setup	35
A.1	Required components	35
A.2	Audio settings	35
A.3	Patch reference	35
	Appendix B Audio Samples	36
B.1	Input samples	36
B.2	Generated samples	36
	Appendix C Configurations and Compatibility Patches	37
C.1	Iteration configuration files	37
C.2	Compatibility patches applied to acids-rave 2.3.1	37
	Appendix D Project Planning	38
D.1	Project Charter	38
D.2	Work Breakdown Structure	39
D.3	Project history and scope reset	40
D.4	Phase schedule	40

List of Figures

3.1	Block diagram of the real-time voice-to-instrument resynthesis pipeline. The Pure Data host invokes the BRAVE causal autoencoder via the <code>nn~</code> external; the encoder maps each block of audio $x(t)$ to a 16-dimensional latent z and the decoder regenerates $\hat{x}(t)$ in the target-instrument timbre. Latency contributions per stage are listed in Table 3.1.	10
5.1	KL divergence trajectory for guitar_v1 during Phase 1 training. The KL term falls from approximately 0.83 at step zero to 0.25 by step 1.24 M. A decreasing KL coupled with a decoder forward difference of 0.006 (below the 0.01 health threshold) constitutes the diagnostic signature of posterior collapse (mode 1 of the failure taxonomy, §5.4).	22
5.2	Adversarial dynamics across the three Phase-2 iterations: guitar_v4 exhibits a widening GAN logit gap (peaks at 5.65, discriminator dominance); guitar_v5 collapses to a narrow gap of approximately 0.80 (discriminator collapse); guitar_v6 is the first checkpoint where the gap reverses direction during Phase 2, indicating an approximate adversarial equilibrium.	23
5.3	STFT magnitude spectrograms (log scale) for an in-distribution GuitarSet sample: input (left) and the canonical guitar_v6 reconstruction (right). The reconstructed spectrogram preserves the harmonic structure and onset patterns of the input although it lacks fine spectral detail above approximately 4 kHz.	24
5.4	drums_v2 GAN logit gap over the duration of training. Phase 1 (steps 0–1.5 M) maintains a near-zero adversarial gap. From Phase 2 onset onward, the gap widens monotonically from approximately 9.8 to over 20, reproducing the discriminator-dominance pattern previously observed in guitar_v3/v4 despite the use of <code>update_discriminator_every=4</code>	26
5.5	Per-source noise-floor distribution across the source recordings of the consolidated guitar corpus (boxplots; $n = 360, 52, 128, 128, 123, 128$ files respectively). Median noise floors range from -82.4 dBFS (IDMT Career SG DI) to -46.5 dBFS (GuitarSet mic-captured acoustic), a 35.9 dB spread that exceeds typical within-performance dynamic variation.	29

List of Tables

2.1	A concise comparison of the architectures analysed in §2.1. Latency is the end-to-end audio-to-audio delay as measured and reported by respective authors on their cited hardware configurations; entries without a directly measurable latency are qualitatively discussed.	6
3.1	Estimated latency decomposition for the deployed pipeline (block size 64 samples at 44.1 kHz, host environment Pure Data 0.55 on Windows 11 with the WASAPI driver). Block-size and model-forward values are computed analytically from configuration parameters; driver-buffer values are reported by the WASAPI configuration and observed informally during live testing rather than measured through a calibrated loopback test. Formal loopback-based latency measurement is identified as future work in Chapter 6.	11
3.2	Composition of the guitar training corpus after format filtering (mono, 16-bit PCM, 44.1 kHz). The IDMT-SMT-Guitar dataset 2 was excluded because its 24-bit files were incompatible with the LMDB construction stage, as discussed under the preprocessing pipeline. After preprocessing, the resulting LMDB reports <code>n_seconds</code> = 57665 (~16 h); the higher figure reflects overlapping segmentation rather than additional source audio.	13
4.1	Hardware and software environment used for all training and deployment in this project.	18
4.2	Configuration evolution across guitar iterations and the two drums iterations. Each row identifies the main change introduced and the failure mode it addressed.	19
5.1	Summary of failure modes across the guitar development iterations.	23
5.2	Cross-checkpoint calibration of the STFT-magnitude-correlation diagnostic. All values are means across $N = 8$ GuitarSet samples. Output RMS reflects the energy of the model output relative to the input (mean input RMS ≈ 0.065).	25
5.3	Per-source noise-floor distribution across the source recordings of the consolidated guitar corpus (919 files; see text for the relationship to the 899-file training corpus of Table 3.2). The 5th-percentile RMS in 100 ms windows estimates the noise floor of each recording.	28
D.1	Three-phase schedule of the active iteration cycle (24 March – 12 June 2026), following the UPF TFG guide (Hernández-Leo et al., 2012). December 2025 – 23 March 2026 was an earlier scoping and first (unsuccessful) environment-setup phase, described above. Phase boundaries correspond to the dated training runs in the project repository.	40

1. Introduction

1.1 Motivation

Currently, in the world of live electronic music performance, there is a tendency to use deep neural models to control timbre in a very expressive way. The human voice is a very expressive, natural and simple way to control such systems and does not need any special equipment except a microphone.

Neural audio synthesis is now advancing thanks to key models such as WaveNet (van den Oord et al., 2016), NSynth (Engel et al., 2017), and DDSP (Engel et al., 2020). However, these initial models are not very suitable for live use. Autoregressive models require a lot of computing power, and on the other hand, early differentiable signal processing models often have to focus on explicit tone tracking or processing in windows that can cause noticeable lag.

The launch of RAVE (Caillon & Esling, 2021) became highly influential, facilitating high-quality synthesis even on widely accessible CPU hardware. BRAVE (Caspé et al., 2025) goes a step further with a streaming-causal architecture aimed at creating interactive and responsive music in real-time. However, converting voice to instrument is quite difficult when the source and destination timbres are very different acoustically. For example, plucked string instruments, such as the guitar, have a sharp percussive attack and an exponentially decaying envelope, which is very different from the sustained, formant-rich structure of the human voice. This work investigates whether streaming-causal systems can achieve voice-to-guitar resynthesis that is subjectively plausible as guitar, at a target latency of 10 ms or less so it is usable in live performance. Addressing this cross-modal mismatch is crucial to the enhancement of music interfaces that enable human vocal intuition to be the foundation for machine synthesis.

1.2 Research Questions

The motivation above frames a problem domain rather than a tightly defined investigation. To make the scope of the work explicit and to give the methodology of Chapter 3 and the evaluation of Chapter 5 something concrete to answer, the project is organised around three research questions:

- RQ1** (Feasibility). Can a streaming-causal variational autoencoder of the RAVE/BRAVE family be trained on small, open instrument corpora (under twenty hours per instrument) to operate as a real-time autoencoder at sub-10 ms end-to-end latency on a single consumer GPU, using a reproducible Windows-based toolchain?
- RQ2** (Cross-modal generalisation). Does an instrument-domain-only autoencoder’s learned latent manifold generalise to out-of-distribution vocal excitation at inference time, in the absence of any paired voice-to-instrument training data?
- RQ3** (Failure diagnosis). What training instabilities and degenerate behaviours emerge under these constrained training conditions, and to what extent can each be diagnosed from in-training metrics and synthetic-input probes alone, without recourse to formal perceptual evaluation?

RQ1 and RQ3 are answered constructively across the iteration sequence reported in Chapter 5: a guitar autoencoder that partially reconstructs in-distribution audio is obtained and five distinct training-failure modes are isolated. RQ2 is answered negatively, in a qualified form: instrument-only training did not yield perceptually convincing voice-to-instrument resynthesis on the small open corpora used here, and

the IRCAM percussion baseline (§5.3) suggests this is a training-recipe and data-scale limitation rather than an architectural impossibility.

1.3 Objectives and Scope

To address the research question, this project defines a general objective to design and evaluate a streaming neural resynthesis pipeline for expressive voice-driven instrumental synthesis. The investigation is organised according to the following specific objectives:

- Examine whether it is possible to keep the total latency below 10 ms in a visual programming environment with a causal Variational Autoencoder (VAE) architecture.
- Assess the performance of instrument-specific models when driven by both vocal percussive inputs and sustained tonal singing.
- Document training instabilities and failure modes throughout the RAVE/BRAVE architecture family.
- Evaluate the limits of audio reconstruction fidelity when models are presented with out-of-distribution vocal signals.

The scope of this project is restricted to monophonic audio resynthesis since this constraint lowers computational complexity and also enables stable inference on consumer-grade devices. Consequently, polyphonic modelling, offline batch processing, and linguistic speech conversion (text-to-speech) are considered out-of-scope.

1.4 Methodology

The approach combined iterative system implementation with structured diagnostic protocols. The methodology, detailed in Chapter 3, is organised around four operational axes — system architecture, training paradigm, dataset preparation, and diagnostic protocols — operationalised through the following five processes:

1. **Dataset preparation:** Public guitar and drum-kit corpora were preprocessed to a uniform 44.1 kHz mono format. This stage included a quantitative analysis of noise-floor heterogeneity to identify potential biases in the training data.
2. **Training procedure:** Models were trained under a two-step procedure: an independent representation learning phase optimising a multi-scale STFT loss, followed by adversarial fine-tuning intended to improve perceptual naturalness.
3. **Deployment:** The trained TorchScript models were integrated into Pure Data through the `nn-external`, supporting live vocal interaction at a 64-sample block size.
4. **Quantitative evaluation:** The main focus was using end-to-end latency benchmarks together with synthetic-input diagnostic probes (§3.4). The degree of alignment between input and output Short-Time Fourier Transform (STFT) magnitudes was also considered as a metric for spectral reconstruction closeness.
5. **Qualitative testing:** Listening evaluation at each checkpoint was conducted informally by the author. No formal MUSHRA or MOS test with multiple participants was administered; the implications of this evaluation design are stated explicitly in §3.4 and §3.5 rather than deferred to the conclusions.

1.5 Original Contributions

The following are the original contributions of this work. Each is grounded in the experimental evidence reported in Chapter 5 and the methodological framework of Chapter 3:

- **Original contribution 1 — Failure-mode taxonomy.** A systematic, evidence-anchored account of five distinct training-failure modes observed across an eight-iteration training sequence: posterior collapse (v1), latent underutilisation (v2), discriminator dominance / generator mode collapse (v3, v4, drums_v2), discriminator collapse (v5), and out-of-distribution decoder behaviour (v6, drums_v1). The author is not aware of a published RAVE/BRAVE study that traces this many distinct failure modes within a single training programme.
- **Original contribution 2 — Calibration of the STFT-magnitude-correlation diagnostic.** A cross-checkpoint calibration study (§5.1.4, Table 5.2) showing that the STFT-magnitude-correlation metric initially proposed as a working acceptance heuristic does not robustly discriminate working autoencoders from collapsed ones, because collapsed checkpoints produce trivially correlated near-silent outputs. This is a methodological finding that constrains how the metric should be used in future replication work.
- **Original contribution 3 — External-baseline disentanglement.** A side-by-side diagnostic comparison between drums_v1 and the official ACIDS-IRCAM percussion checkpoint (§5.3) measured under identical conditions in Pure Data, isolating the observed OOD limitation as a training-recipe and dataset-scale issue rather than an architectural impossibility.
- **Original contribution 4 — Partially-reconstructing in-distribution guitar autoencoder.** A streaming-causal RAVE v2 guitar autoencoder (guitar_v6_best.ts) recovered from a sequence of failed iterations, which partially reconstructs in-distribution guitar audio and serves as the reference point against which cross-modal vocal input is evaluated.
- **Original contribution 5 — Dataset-heterogeneity quantification.** A noise-floor analysis (§5.5, Table 5.3) measuring a 35.9 dB spread across six recording paths within the consolidated guitar corpus, with documented implications for what features the encoder is likely to internalise as salient.
- **Original contribution 6 — Open, reproducible Windows pipeline.** A version-controlled training, evaluation, and deployment pipeline (preprocessing, four acids-rave compatibility patches, training launchers, diagnostic and figure-generation scripts, Pure Data patches) released alongside this thesis so that the eight reported iterations can be re-run on commodity Windows hardware.

1.6 Document Structure

The thesis is organised into six chapters. Chapter 2 reviews the state of the art in neural synthesis and timbre transfer. Chapter 3 details the methodology, focusing on system architecture and training protocols. Chapter 4 describes the implementation of the Pure Data environment and the software pipeline. Chapter 5 presents the evaluation results, including the failure taxonomy, the external-baseline validation, and the dataset heterogeneity analysis. Chapter 6 summarises the findings and proposes future work across calibrated latency measurement, formal perceptual evaluation, RAVE v3 architectures, mixed-domain training data, hybrid pipelines, and replication on larger corpora. The Pure Data demonstration setup is documented in Appendix-A; reference audio samples are listed in Appendix-B; configuration files and the four compatibility patches in Appendix-C; and the project planning artefacts (Project Charter, Work Breakdown Structure, phase schedule) in Appendix-D.

2. State of the Art

Over the past decade, the field of neural audio synthesis has shifted from slow autoregressive models toward streaming-causal architectures capable of producing music in real time. This chapter is structured around three topics: (i) the architectural genealogy from WaveNet to BRAVE, (ii) methods of timbre transfer and voice conversion, and (iii) datasets used to train and test these models. The chapter finishes by clearly indicating the gaps in research that the present work aims to fill.

Literature selection methodology. The works discussed have been chosen based on three factors: (a) direct architectural relevance to the streaming-causal Variational Autoencoders (VAEs) used in this work, (b) methodological relevance to cross-modal timbre transfer, and (c) additional contributions to GAN stability and audio reconstruction quality that influenced the RAVE v2 architecture. Priority was given to highly ranked conference and journal venues (NeurIPS, ICML, ICLR, ISMIR, ICASSP) and arXiv preprints were used when they are the canonical reference for a widely used architecture. The literature search was conducted from March to May 2026. As a limitation of this survey, streaming neural audio synthesis is a fast-evolving field; some very recent architectures are mainly documented through preprints and open-source implementations rather than peer-reviewed publications, and the canonical citation of several systems used in this project (most notably BRAVE) is an arXiv preprint.

2.1 Neural Audio Synthesis Architectures

Early Neural Synthesis Architectures

WaveNet (van den Oord et al., 2016), an autoregressive model factorising the joint probability of the waveform into a product of conditional probabilities and generating audio sample by sample, laid the groundwork for modern neural audio synthesis. WaveNet achieved levels of naturalness in text-to-speech tasks that were unprecedented at the time, but live usage is impractical because every single audio sample requires a full forward pass through the network. NSynth (Engel et al., 2017) applied the same architectural family to musical notes by training a WaveNet autoencoder on more than 300,000 one-note sound samples. The authors demonstrated a 16-dimensional temporal embedding representation that captures the semantic timbral qualities of sounds and allows morphing from one instrument identity to another. NSynth still inherits the autoregressive bottleneck of WaveNet and is restricted to isolated notes rather than continuous performance.

Differentiable Signal Processing

Differentiable Digital Signal Processing (DDSP) (Engel et al., 2020) introduced a different perspective. Rather than outputting raw audio samples, DDSP places conventional DSP modules (harmonic additive synthesisers and filtered noise) directly into the computation graph and trains a neural network to produce their control parameters. The inductive bias of these synthesisers leads to significant reductions in data and model-size requirements compared with autoregressive models. The trade-off is that DDSP depends on explicit fundamental-frequency (F_0) and loudness signals to perform inference; in a live performance setting, this entails an ongoing dependence on an external pitch-estimation module on the critical path.

Variational Autoencoders for Raw Audio: The RAVE Architecture

Whereas WaveNet and DDSP improve reconstruction by operating directly on the raw waveform, the Variational Autoencoder model (Kingma & Welling, 2013) suggests learning structured latent variables usable both for accurate reconstruction and for new data generation. RAVE (Caillon & Esling, 2021) brings this theory into practice for real-time audio.

Two-stage training pipeline. RAVE first trains a VAE and then adversarially fine-tunes it. In Phase 1, a multi-scale STFT loss trains the encoder–decoder pair to minimise spectral reconstruction error (Caillon & Esling, 2021; Engel et al., 2020). In Phase 2, the encoder is frozen and the decoder is trained to fool a multi-scale waveform discriminator (Goodfellow et al., 2014; Kong et al., 2020), enhancing the perceptual quality of the output.

Multi-band decomposition for real-time inference. To enable real-time inference on typical CPU hardware, RAVE uses a 16-band frequency decomposition based on a Pseudo-Quadrature Mirror Filter (PQMF) bank (Cruz-Roldán et al., 2002). This decomposes the signal into sixteen subbands, effectively reducing the per-band processing rate prior to the encoding step (Caillon & Esling, 2021).

Adversarial stabilisation techniques. Stability of the adversarial phase depends on additional techniques from the generative-modelling literature: spectral normalisation of the discriminator (Miyato et al., 2018), multi-period discriminators inherited from HiFi-GAN (Kong et al., 2020), and multi-scale spectral losses computed over different STFT window sizes (Engel et al., 2020). Together, these strategies have established RAVE as one of the most widely adopted neural timbre-transfer architectures for interactive applications.

Streaming-Causal Extension: BRAVE

Standard RAVE operates with non-causal convolutional layers, and its end-to-end latency was measured at tens of milliseconds (Caillon & Esling, 2021), a delay likely to disrupt live musical performance. BRAVE (Caspé et al., 2025) redesigns RAVE for streaming inference. The most important architectural modifications are: replacing non-causal convolutions with causal ones, reducing the encoder’s compression factor to limit accumulated group delay, and relaxing the PQMF attenuation specification to shorten the filter taps. Causal convolutions are the central enabler: by removing the requirement of future temporal context, they allow inference to run stepwise on streaming audio without buffering ahead of the current sample. Combined with cached convolutional state and a specialised C++ inference path, these modifications achieve a reported audio-to-audio latency below 10 ms while preserving the qualitative characteristics of RAVE (Caspé et al., 2025).

SoundStream (Zeghidour et al., 2022), a neural codec developed in parallel, independently showed that convolutional encoder–decoder topologies could be used for low-latency streaming when padding schemes are designed carefully, providing architectural justification for BRAVE’s causal design. A trade-off acknowledged by the BRAVE authors is that the limited temporal context can degrade pitch accuracy at extreme registers for melodic instruments.

Diffusion-Based Audio Synthesis

The past two years have seen the emergence of latent diffusion architectures adapted to general audio and music generation, including AudioLDM (Liu et al., 2023), MusicGen (Copet et al., 2023), MAGNeT (Ziv et al., 2024) and Stable Audio (Evans et al., 2024). These models achieve state-of-the-art reconstruction quality and have substantially expanded the range of text-conditioned and inpainting tasks that can be demonstrated on contemporary hardware. They are excluded from the present comparison for two reasons. First, their inference cost is typically too computationally expensive for sub-10 ms interactive deployment, even with accelerated samplers, which precludes streaming use. Second, their conditioning

interfaces — natural-language prompts, instrument-class labels or melody contours — are orthogonal to the continuous, voice-driven control that motivates the present project. Diffusion-based approaches remain an open research direction for offline cross-modal resynthesis and are revisited as future work in §6.4.

Synthesis: An Architectural Evolution Shaped by Latency

Considered consecutively, these architectures form a distinct evolutionary path. WaveNet was the first to show that a neural network could generate waveforms directly and realistically, but it could only operate offline. DDSP enabled real-time inference by giving up the freedom of generating raw waveforms in favour of an inductive bias toward DSP-based synthesis, but at the cost of requiring an external pitch-estimation stage. RAVE removed the need for explicit pitch conditioning by discovering the latent space directly, but its non-causal context kept latency above the level of perceptual immediacy for live use. BRAVE brings inference down to the sub-10 ms regime by accepting an even smaller temporal context, the latest step in a decade-long progression away from offline waveform modelling and toward interactive musical instruments. Diffusion-based models represent a parallel branch oriented toward offline high-quality generation rather than real-time interaction.

Project architectural choice. BRAVE is the only architecture among those reviewed that simultaneously meets the three major constraints of the project: implicit pitch modelling (eliminating the need for an external pitch tracker on the audio path), streaming inference, and sub-10 ms end-to-end latency. WaveNet and NSynth are excluded by their autoregressive nature; DDSP by its reliance on external pitch tracking; standard RAVE by its non-causal context; and the diffusion family by inference cost. The methodology and assessment in Chapters 3 and 5 are therefore based on BRAVE. Table 2.1 summarises the comparison.

Table 2.1: A concise comparison of the architectures analysed in §2.1. Latency is the end-to-end audio-to-audio delay as measured and reported by respective authors on their cited hardware configurations; entries without a directly measurable latency are qualitatively discussed.

Model	Real-time	Causal	Latency	Pitch ing	condition- ing	Latent representa- tion	Main limitation for live voice use
WaveNet (2016)	No	Yes	Minutes per second of audio	Optional (linguistic)	(linguistic)	None (direct waveform)	Sample-by-sample generation prohibits live use
NSynth (2017)	No	Yes	Minutes per second of audio	None		16-D temporal embedding	Restricted to isolated notes
DDSP (2020)	Yes	No	Real-time; bounded by pitch tracker	Explicit F0 + loudness		Explicit DSP control parameters	External pitch tracker on critical path
TimbreTron (2018)	No	No	Offline (CQT + WaveNet vocoder)	Implicit via CQT		CQT spectrogram	Spectrogram-domain inference too slow
RAVE (2021)	Yes	No	~50 ms reported	Implicit		Learned latent	VAE Non-causal context limits perceptual immediacy
BRAVE (2025)	Yes	Yes	< 10 ms reported	Implicit		Learned latent (causal)	VAE Shorter temporal context can degrade pitch at extremes
Diffusion family (2023–2024)	No	No	Seconds to minutes per second of audio	Text / melody	class /	Latent diffusion	Inference cost prohibits streaming use

Pretrained ACIDS-IRCAM checkpoints. Beyond the published architectural descriptions surveyed above, the ACIDS-IRCAM group distributes a library of pretrained RAVE-family checkpoints (percussion, darbouka, voice, violin, electric guitar, and others) accessible through the ACIDS RAVE-VST API (ACIDS-IRCAM, 2024). These checkpoints were trained on larger and, where reported, more curated corpora than the open datasets reviewed in §2.3, although their exact training configurations are not published alongside the released weights. In Chapter 5 the official percussion checkpoint is used as an external baseline against which the responsiveness of the present project’s drum-trained model is calibrated, in order to disentangle architectural from training-recipe contributions to the out-of-distribution voice limitation observed in this work.

2.2 Timbre Transfer and Voice Conversion

The trajectory in §2.1 is concerned with creating audio from a learned distribution. Converting one sound into the timbre of another has a related but distinct literature that predates RAVE and that nonetheless points to the cross-modal problems addressed by this thesis.

Definition of Timbre

Classically, timbre is the perceptual characteristic that differentiates two sounds having the same pitch and loudness. It is usually treated as a multidimensional notion encompassing spectral envelope, attack and decay characteristics, fine-grained temporal modulations, and the resonant properties of the source. In machine learning, timbre transfer is the conversion of the timbral features of a source signal into those of a target instrument while preserving musical elements such as pitch, rhythm, and dynamics.

Spectrogram-Domain Transfer

The first demonstrations of high-quality musical timbre transfer were given by TimbreTron (Huang et al., 2018), which uses CycleGAN, an image-to-image translation network, on constant-Q transform (CQT) spectrograms. CQT representations have approximate pitch equivariance, which makes them suitable for music. TimbreTron solves the phase-reconstruction problem associated with spectrogram-domain approaches by attaching a conditional WaveNet vocoder to convert the transferred CQT to a waveform. The method produces convincing piano-to-harpsichord and similar conversions but cannot be used live because of WaveNet’s offline inference time.

Voice Conversion

Voice conversion (VC) is concerned with changing the speaker identity while preserving the spoken content. AutoVC (Qian et al., 2019) proposed an autoencoder-only model in which an information bottleneck separates speaker-dependent features from phonetic content, making zero-shot transfer possible even for target speakers not seen during training. Neural audio codecs such as SoundStream (Zeghidour et al., 2022) take the encoder–decoder framework further by performing general audio compression with residual vector quantisation, achieving very high reconstruction quality at low bitrates.

Vocal Percussion Transcription

A parallel line of work concerns the discrete classification of beatbox and other vocal percussion sounds into drum-instrument categories (kick, snare, hi-hat, cymbal). Early systems relied on spectral and timbral features combined with classical machine-learning classifiers (Stowell & Plumbley, 2008). Subsequent

work introduced labelled corpora — most notably the Amateur Vocal Percussion (AVP) dataset (Delgado et al., 2019), which provides 9780 onset-aligned, four-class vocal-percussion utterances from 28 participants — and shifted to convolutional networks operating on short audio windows. Vocal-percussion transcription is methodologically orthogonal to the timbre-transfer task that frames the present thesis: it produces symbolic onset-class labels rather than a continuous resynthesised waveform. The two approaches are, however, naturally combinable into hybrid pipelines that route classified onsets to per-class neural synthesisers or sample banks; this combination is identified as a future-work direction in §6.4.

The Voice-to-Instrument Cross-Modal Gap

The distance between voice-to-voice and voice-to-instrument tasks is substantial. Voice production is characterised by a formant structure generated by the vocal tract and phonetic content communicated through rapid articulation. A plucked-string instrument such as a guitar, on the other hand, derives its sound from mechanical excitation, body resonance and an exponentially decaying envelope; such instruments do not produce formants and have no phonetic content. If a VAE is trained solely on guitars, its latent-space manifold is therefore expected to be poorly aligned with vocal features (Caillon & Esling, 2021; Kingma & Welling, 2013). Whether the encoder can position the voice within a meaningful region of that manifold, and whether the decoder can generate a meaningful output from that positioning, is the empirical question that this thesis addresses through the diagnostic methodology described in Chapter 3.

Synthesis: Timbre Transfer Trades Expressiveness against Real-Time Tractability

The studies above highlight a continuing dilemma in this area: expressiveness is best served by unconstrained raw-waveform models such as TimbreTron, whereas real-time tractability favours encoder-decoder bottlenecks like AutoVC and RAVE. BRAVE represents an extreme on this trade-off, willing to adopt strong inductive biases (causal convolutions, PQMF compression, fixed bottleneck) so as to carry out inference at a reported latency of less than 10 ms. The same property that enables real-time use also imposes severe limitations on the model’s ability to ingest and reconstruct acoustic structure from a source very different from its training distribution — exactly the scenario that the voice-to-guitar task in this thesis opens up.

2.3 Neural Audio Synthesis Datasets

The performance of RAVE-family models strongly correlates with the quantity, variety and uniformity of training data. A duration of three to fifty hours of single-source audio is recommended (Caillon & Esling, 2021).

GuitarSet. (Xi et al., 2018) contains acoustic guitar recordings of around three hours from six professional players in five musical styles. Each performance is recorded with both a primary microphone and a hexaphonic pickup that isolates each string. Annotations include pitch contours, beat positions, chord labels and playing techniques. Although widely used as a music information retrieval benchmark, the generalisation capacity of models trained on this dataset alone is limited by its small duration and single-instrument nature.

Guitar-TECHS. (Pedroza et al., 2024) is a multi-angled electric-guitar set of approximately 5.2 hours covering six playing techniques. Each performance is recorded simultaneously through a direct-input (DI) channel, a close-miked amplifier and stereo binaural microphones. The DI channel is preferred for RAVE training because it provides a clean isolated signal without room acoustics.

IDMT-SMT-Guitar. (Kehling et al., 2014) adds timbral diversity through six guitars (five electric, one acoustic) played by three musicians, recorded through both microphone and DI capture paths. The dataset is divided into isolated notes (datasets 1, 2) and continuous performance excerpts (datasets 3, 4); only the latter are relevant for the current work.

Groove MIDI Dataset. (Gillick et al., 2019) provides a percussive analogue to the guitar collections: approximately 13 hours of acoustic drum-kit performances by professional drummers with synchronised MIDI and audio across a wide range of styles and tempos. Beyond comfortably exceeding the lower bound for RAVE training, it also supports the secondary case study of this thesis.

Dataset selection rationale and noise-floor heterogeneity. After selection for compatible files (mono, 16-bit, 44.1 kHz), the consolidated guitar corpus (GuitarSet, the Guitar-TECHS DI channel and the continuous-performance subsets of IDMT-SMT-Guitar) is approximately 16 hours. This duration lies within the recommended training window of Caillon and Esling (2021) but introduces a well-known risk: mixing recording paths leads to inconsistency in background noise floors. The quantitative analysis presented in §5.5 reveals a 35.9 dB range between the cleanest direct-input electric recordings and the loudest microphone-captured acoustic recordings. This range exceeds the typical dynamic variation within a single performance and may be encoded as a prominent latent feature, biasing the learned manifold toward recording-method discrimination rather than purely timbral content.

2.4 Research Gap and Motivation

The literature reviewed in this chapter presents a research landscape in which streaming neural audio synthesis has reported the sub-10 ms latency required for live musical interaction (Caspe et al., 2025); in which timbre-transfer methods exist but lose either real-time capability (TimbreTron) or controllability (DDSP); and in which the voice-to-instrument cross-modal task at this latency level remains nearly unexplored. BRAVE has been demonstrated on voice-to-saxophone and beatbox-to-drums on smaller corpora, but voice-to-guitar has not been tried, and the BRAVE authors explicitly identify these as open questions.

Most previous contributions emphasise success demonstrations and treat failure as a peripheral observation; here, failures are treated as worthy objects of analysis. The three main issues this thesis addresses are:

- **The pitch realisation of plucked-string instruments.** A saxophone can sustain a sound by modulating breath pressure, whereas a guitar note is struck sharply and decays exponentially. The interplay between such temporal dynamics and a streaming-causal encoder with restricted temporal context is one of the issues not covered in earlier BRAVE work (Caspe et al., 2025).
- **The acoustic difference between voice and guitar.** Voice formants and guitar body resonance occupy substantially different regions of acoustic feature space. Whether a VAE manifold trained only on guitar can be steered to accept voice as an in-distribution input, or whether voice is naturally an outlier of the trained manifold, has not been answered quantitatively in the prior literature.
- **Training on multiple sources under very different recording conditions.** The consequence of mixing acoustic and electric guitar recordings, with the 35.9 dB noise-floor spread reported in §5.5, for BRAVE’s latent space is the topic of this work and has not been addressed by prior studies.

These observations serve as the foundation for the present investigation of real-time voice-to-instrument cross-modal conversion. They motivate the methodological decisions in the choice of BRAVE for deployment and the experimental design presented in the next chapter.

3. Methodology

This chapter details the technical setup, training routines, and evaluation mechanisms used to explore the research question of this thesis: whether a causal streaming neural autoencoder can perform voice-driven instrument resynthesis perceptually close to the target instrument at sub-10 ms latency, and what training instabilities and failure modes emerge when this task is performed on heterogeneous, limited-size datasets. The methodology is organised around four operational axes — system architecture, training paradigm, dataset preparation, and diagnostic protocols — each linked to the corresponding evaluation in Chapter 5.

3.1 System Architecture

The real-time resynthesis pipeline operates as a low-latency end-to-end stream from acoustic input to digital output. The signal path is linear: an acoustic input is captured by a microphone, converted through an analogue-to-digital converter (ADC), processed by the BRAVE model in a host environment, and returned through a digital-to-analogue converter (DAC) to the speakers (Figure 3.1).

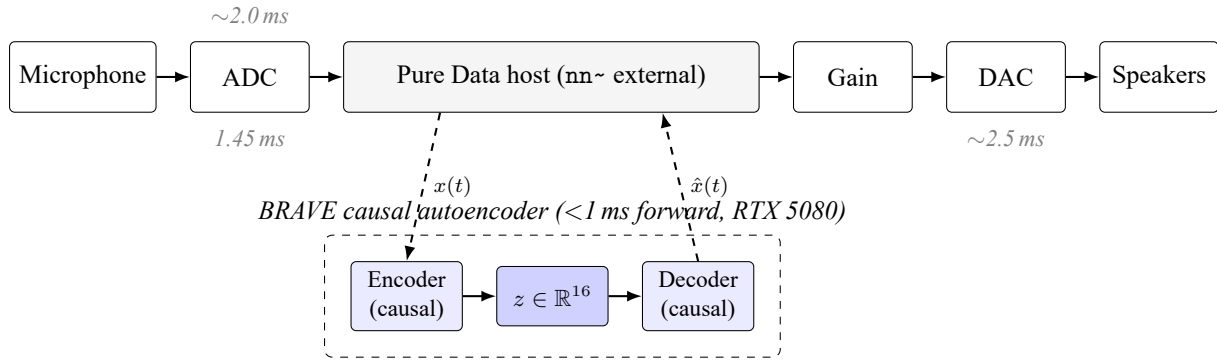


Figure 3.1: Block diagram of the real-time voice-to-instrument resynthesis pipeline. The Pure Data host invokes the BRAVE causal autoencoder via the `nn~ external`; the encoder maps each block of audio $x(t)$ to a 16-dimensional latent z and the decoder regenerates $\hat{x}(t)$ in the target-instrument timbre. Latency contributions per stage are listed in Table 3.1.

The principal design constraint is an end-to-end latency below 10 ms, which lies within the approximate range identified by Wessel and Wright (2002) as the threshold for intimate musical control. The system uses Pure Data (Pd) combined with the `nn~ external` (Caillon & Esling, 2021).

Design rationale: host environment. Pure Data and `nn~` were chosen after evaluating an alternative based on a custom C++ audio engine using the Torch C++ API (LibTorch). Although a custom engine offers higher optimisation potential, it would have required substantial development time for audio I/O and interface management. Pure Data was adopted because it is a stable deployment target for RAVE-family models (Caillon & Esling, 2021) and provides immediate access to standard DSP operations (gain multipliers, limiters) that proved essential given the low raw output level of early model iterations.

Architectural backbone: streaming-causal RAVE. The synthesis engine is BRAVE (Bravely Real-time Audio Variational autoEncoder). Whereas standard RAVE uses non-causal convolutions and reports a buffering delay of approximately 50 ms (Caillon & Esling, 2021), BRAVE adopts a streaming-causal

layout (Caspe et al., 2025). By restricting the model’s temporal context to current and past samples, latency is determined almost entirely by the processing block size and the cost of a single model inference. Table 3.1 breaks down the estimated end-to-end latency.

Table 3.1: Estimated latency decomposition for the deployed pipeline (block size 64 samples at 44.1 kHz, host environment Pure Data 0.55 on Windows 11 with the WASAPI driver). Block-size and model-forward values are computed analytically from configuration parameters; driver-buffer values are reported by the WASAPI configuration and observed informally during live testing rather than measured through a calibrated loopback test. Formal loopback-based latency measurement is identified as future work in Chapter 6.

Component	Approximate latency
Audio driver buffer (input)	~2.0 ms
ADC + Pure Data block (64 samples @ 44.1 kHz)	1.45 ms
BRAVE model forward pass (per block, RTX 5080)	< 1.0 ms
Pure Data output buffer + DAC	~2.5 ms
Total end-to-end (estimated)	~ 7 ms

The estimated total of approximately 7 ms falls within the latency range discussed by Wessel and Wright (2002).

3.2 Training Paradigm

Experimental premise: unpaired cross-modal resynthesis. Before describing the two-phase training procedure, it should be stated explicitly that the system is not trained on paired voice-to-instrument examples. BRAVE is trained as an instrument-domain autoencoder, using only guitar or drum recordings; vocal input is introduced only at inference time, under the working assumption that the encoder’s learned latent manifold will generalise to out-of-distribution (OOD) vocal excitation signals. This assumption is motivated by prior observations that neural audio autoencoders often organise latent spaces around general spectral-temporal structure rather than strictly source-specific categories. The empirical investigation of whether this assumption holds for guitar and drums — and the failure modes that emerge when it does not — constitutes a substantial part of the contribution presented in Chapter 5.

Training is divided into two phases that decouple the learning of a compact latent representation from the synthesis of perceptually plausible waveforms.

Phase 1: representation learning. In the first phase, the model operates as a standard Variational Autoencoder (Kingma & Welling, 2013). The encoder and decoder are trained jointly to minimise a multi-scale Short-Time Fourier Transform (STFT) reconstruction loss comparing magnitude spectrograms of input and output at multiple temporal resolutions (Caillon & Esling, 2021; Engel et al., 2020). The spectral loss is motivated by perceptual considerations: subtle phase variations are largely inaudible to human listeners, so the model is not penalised for them. The latent space is regularised through a Kullback–Leibler (KL) divergence term that encourages the posterior distribution to match a standard Gaussian prior (Kingma & Welling, 2013).

Phase 2: adversarial fine-tuning. Once the reconstruction loss converges, the encoder is frozen and the decoder is fine-tuned as the generator component of a Generative Adversarial Network (Goodfellow et al., 2014). Freezing the encoder is consistent with established RAVE practice (Caillon & Esling, 2021) and aims to stabilise the latent representation learned in Phase 1 by preventing adversarial gradients from destabilising the bottleneck structure. A discriminator network distinguishes between real audio samples

and the decoder’s reconstructions. The adversarial objective is intended to recover high-frequency textural detail and transient information that the GAN-vocoder literature reports as smoothed over by purely spectral losses (Kong et al., 2020).

Design rationale: beta scheduling. *Problem.* In the first training iteration (guitar_v1), a constant beta of 0.1 was applied from step zero, as inherited from the unmodified `c128_r10.gin` configuration shipped with the BRAVE repository. This configuration produced posterior collapse, a well-documented training instability in which the latent space becomes uninformative because KL pressure is applied before the model has learned a meaningful reconstruction (Bowman et al., 2016).

Theoretical motivation. A recognised stabilisation strategy in the VAE literature is to anneal the KL term gradually, allowing the encoder to discover salient input features before being forced to compress them toward the prior (Bowman et al., 2016).

Adopted solution. All subsequent runs (v2–v6) employ a log-space warmup in which beta initialises at 1×10^{-4} , providing effectively zero pressure, and ramps exponentially to 0.1 over the first one million steps.

Observed outcome. No posterior collapse was observed in subsequent runs, and Phase 1 produced stable rising KL trajectories in all five iterations using the warmup schedule, as documented in Chapter 5.

Design rationale: latent dimensionality. The latent size was initially set to 128, the RAVE default. Diagnostic results for guitar_v2 showed a utilisation of only approximately 0.005 nats per dimension, indicating that the latent space was substantially larger than the dataset could support. The latent dimensionality was reduced to 16, matching established practice in the published RAVE community for guitar-scale datasets. This reduction appeared to encourage more efficient latent utilisation in later iterations, which achieved approximately 0.04 nats per dimension.

Training configuration. All guitar iterations use the Adam optimiser at the default RAVE learning rate, with the beta schedule defined per iteration in the corresponding `.gin` configuration file. The training batch size is 8 throughout. Phase 1 spans 1 M steps in iterations v1–v4 and is extended to 1.5 M in v5 and v6 to allow a longer reconstruction phase before adversarial training begins. Each full run targets approximately 3 M total steps.

Hardware environment. Training was performed on a single workstation: NVIDIA RTX 5080 GPU (16 GB VRAM), Windows 11, Python 3.14, PyTorch 2.11.0+cu128, acids-rave 2.3.1, pytorch-lightning 2.6.1. Approximately 90 to 120 wall-clock hours are required per full run at an observed throughput of 9–12 optimisation steps per second.

Checkpoint selection. Checkpoints are saved every 10,000 training steps. RAVE’s built-in `best.ckpt` mechanism tracks the lowest validation reconstruction loss across each run segment. For iterations in which the final-epoch model degraded after Phase 2 (notably guitar_v6), the canonical checkpoint was selected manually based on listening comparison against intermediate epochs, as detailed in Chapter 5.

Reproducibility. All configuration files, preprocessing scripts and evaluation utilities used in the experiments were version-controlled throughout development and will be released alongside the thesis as a public Git repository, together with the diagnostic Python scripts and the Pure Data deployment patches.

3.3 Datasets and Preprocessing

The performance of RAVE-family models depends heavily on the quantity, diversity and homogeneity of training data. This section documents the corpora used, the rationale for their inclusion, and the preprocessing pipeline. The scale of the corpora is modest by contemporary generative-audio standards but reflects realistic availability of open instrument datasets, particularly for guitar.

Guitar corpus. The guitar corpus combines three public sources after filtering for compatible audio formats. GuitarSet contributes mic’d acoustic recordings with rich performance variation; Guitar-TECHS contributes clean electric direct-input (DI) signals isolated from amplifier and room colouration; IDMT-SMT-Guitar dataset 4 contributes timbral diversity across multiple electric and acoustic guitars and capture paths. This combination expands the model’s exposure to guitar timbres while accepting heterogeneity in noise floor and spectral character. The consequences of this trade-off are quantified in the noise-floor analysis (§5.5).

Table 3.2: Composition of the guitar training corpus after format filtering (mono, 16-bit PCM, 44.1 kHz). The IDMT-SMT-Guitar dataset 2 was excluded because its 24-bit files were incompatible with the LMDB construction stage, as discussed under the preprocessing pipeline. After preprocessing, the resulting LMDB reports `n_seconds` = 57665 (~16 h); the higher figure reflects overlapping segmentation rather than additional source audio.

Source	Recording method	Raw duration	File count
GuitarSet (Xi et al., 2018)	Mic’d acoustic, both mic positions	~3.05 h	360
Guitar-TECHS DI (Pedroza et al., 2024)	Electric DI	~1.5 h	32 (post stereo filtering)
IDMT-SMT-Guitar dataset 4 (Kehling et al., 2014)	Mixed: acoustic_mic, acoustic_pickup, electric DI	~4.35 h	507
Total raw	—	~9 h	899

Drums corpus. The drums corpus is drawn from the Groove MIDI Dataset (Gillick et al., 2019), totalling approximately 10.86 hours after stereo-to-mono conversion. Groove substantially exceeds the dataset scales used in several published RAVE and BRAVE demonstrations, reducing the risk of dataset-size-induced training instabilities; the inclusion of diverse tempos and rhythmic patterns across multiple professional drummers is also designed to improve the robustness of the learned latent representation across transient-rich percussive events.

Preprocessing pipeline. A five-stage Python pipeline was implemented using `librosa` and `soundfile`:

1. **Validation.** Each input file is loaded with `librosa.load`. Files that fail to load or contain no audio are discarded.
2. **Mono conversion.** Stereo files are downmixed to a single channel.
3. **Standardisation.** Audio is resampled to 44.1 kHz and saved as 16-bit PCM WAV.
4. **Artefact filtering.** Metadata artefacts (notably `__MACOSX/` .wav files inside Mac-archived datasets) and 24-bit files are removed. A preprocessing validation stage was incorporated after empirical observation that incompatible 24-bit audio files caused instability in the standard `rave preprocess` pipeline.
5. **LMDB construction.** The processed WAV files are pre-decoded into a Lightning Memory-Mapped Database (LMDB) using `rave preprocess --no-lazy`, reducing disk I/O overhead during training.

Noise-floor analysis. To characterise the heterogeneity of the guitar corpus, a noise-floor analysis was performed. For each file, the Root Mean Square (RMS) energy was computed in non-overlapping 100 ms windows, and the 5th percentile of these RMS values was taken as the noise-floor estimate, converted to dBFS. Per-source aggregates (median and inter-quartile range) characterise the spread across the corpus. The full results, presented in §5.5, show a 35.9 dB spread between the cleanest electric DI sources and the loudest microphone-captured acoustic sources, which informs the interpretation of latent-space behaviour later in the thesis.

Task-specific difficulty considerations. Beyond dataset composition, the two case studies differ in intrinsic task difficulty in ways that bear directly on the analysis of Chapter 5. Compared with drum resynthesis, guitar resynthesis imposes greater demands on latent continuity and harmonic precision because guitar signals are sustained, pitch-structured, and dominated by stable harmonic relationships, micro-dynamic articulations, and long-duration resonances. Percussive audio, by contrast, is dominated by transient events and broader spectral envelopes that are comparatively forgiving of small latent perturbations. Dataset heterogeneity, latent underutilisation, and adversarial instability are therefore expected to manifest more severely in the guitar case, both because the target signal requires finer-grained representational fidelity and because the cross-modal mapping from voice to guitar spans a larger acoustic gap than voice to drums.

3.4 Diagnostic Methodology

What the evaluation can and cannot establish. Before describing the diagnostic protocols, it is important to state plainly what evaluation evidence this work does and does not produce. The evaluation framework is built from three classes of measurement: (i) in-training scalar metrics tracked through TensorBoard, (ii) synthetic-input responsiveness probes computed on each candidate checkpoint, and (iii) reconstruction comparison on in-distribution audio. These three classes establish whether the optimisation is healthy, whether the encoder differentiates inputs, and whether the decoder produces audible non-degenerate output. They do not establish, by themselves, whether the resulting audio is perceptually convincing. Two design choices in particular shape the strength of the perceptual claims that can be made:

1. **No formal listening test was conducted.** Listening evaluation at each checkpoint was performed informally by the author. No MUSHRA, MOS, ABX, or panel-based comparison with multiple participants was administered. Where the text refers to “working”, “perceptually recognisable”, or “degraded” reconstruction, this reflects single-evaluator judgments rather than population-level perceptual quality.
2. **The STFT-magnitude-correlation criterion is empirical, not validated.** A magnitude-STFT correlation above 0.3 is adopted as a working heuristic, derived from informal listening on a small set of checkpoints rather than from a published normative threshold. §5.1.4 shows that this metric in fact does not robustly discriminate working autoencoders from collapsed ones across the full iteration sequence; output amplitude and listening assessment retain the operational role.

These limitations are not relegated to the conclusions: they constrain how the results in Chapter 5 should be read, and the contribution claimed by this work (Original Contribution 2, §1.5) consists in part of stating and quantifying the limit of the STFT-correlation criterion rather than concealing it. The remainder of this section describes the protocols themselves.

In this thesis, *failure modes* refer to systematic deviations that prevent stable or perceptually meaningful resynthesis. For analytical clarity they are grouped into two overlapping categories: *training instabilities*,

which describe optimisation-dynamic failures observable during training (posterior collapse, discriminator dominance), and *degenerate behaviours*, which describe pathological output characteristics observable after training (latent underutilisation, output amplitude degeneration). The diagnostic framework is designed to detect both categories through in-training diagnostics monitored continuously through TensorBoard, post-training evaluation metrics computed on each candidate checkpoint, and reconstruction validation on in-distribution audio.

In-training diagnostics. During training, five primary TensorBoard scalars are tracked:

- **regularization** (KL divergence): rising values during Phase 1 indicate effective latent-space utilisation; values approaching zero indicate posterior collapse.
- **multiband_spectral_distance**: the primary reconstruction metric. Expected to decrease steadily through Phase 1 and to recover after the transient spike at Phase 2 onset.
- **pred_real** and **pred_fake**: discriminator logits on real and generated audio respectively. Empirically, gaps below approximately 3 between **pred_real** and **pred_fake** were associated with comparatively stable adversarial behaviour in the present experiments, whereas values above 5 frequently preceded discriminator dominance and degraded outputs, consistent with the mode-collapse dynamics described by Goodfellow et al. (2014).
- **loss_dis**: total discriminator loss, used to check whether the discriminator has not collapsed entirely.

Post-training evaluation metrics. For each candidate checkpoint, a Python script runs a synthetic-input responsiveness test using five spectrally distinct inputs: silence; sine waves at 220, 440 and 880 Hz; a linear frequency sweep from 100 to 2000 Hz; and white Gaussian noise. The following criteria are applied:

- **Encoder differentiation.** The pairwise mean absolute difference between latent vectors for distinct inputs must exceed 0.5. This threshold was selected empirically: preliminary testing on collapsed or weakly responsive models produced pairwise latent differences below 0.2, while functionally responsive checkpoints produced values consistently above 0.8.
- **Decoder responsiveness.** The pairwise mean absolute difference between audio outputs must exceed 0.01, derived from the same empirical observation; in the most degenerate cases the pairwise output difference fell below 0.005, corresponding to perceptually indistinguishable outputs.
- **Output amplitude.** The mean absolute amplitude for tonal inputs should fall within $[0.05, 0.3]$, ensuring that the model produces an audible signal without clipping.

Reconstruction validation. A final diagnostic feeds in-distribution guitar audio drawn from GuitarSet through the trained autoencoder. Two measures are computed: the input/output amplitude ratio (target close to 1.0) and the magnitude-STFT correlation between input and output. As no widely standardised threshold exists in the published literature for “perceptually acceptable” neural audio reconstruction expressed as STFT magnitude correlation, this thesis adopts a value above 0.3 as an empirical heuristic, derived from informal listening comparisons against checkpoints with lower correlations. This is treated as a pragmatic working threshold rather than a normative claim, and its limitations are quantified by the cross-checkpoint calibration in §5.1.4.

Validity limitations of the diagnostic framework. Four caveats apply to the framework presented above and are acknowledged here rather than relegated to the conclusions.

Firstly, synthetic sine-wave probing is a structural test of input differentiation and does not in itself establish perceptual realism of the output; it only verifies that the network’s internal mappings are non-degenerate.

Secondly, the STFT magnitude correlation criterion is subject to two interrelated limitations. First, it measures only spectral envelope similarity and does not account for phase-dependent effects, micro-temporal modulations, or other features known to influence perceptual quality. Second, cross-checkpoint calibration reported in §5.1.4 reveals that the metric does not robustly discriminate working autoencoders from collapsed ones: near-silent outputs from collapsed checkpoints exhibit STFT magnitudes whose general spectral shape (low-frequency dominance, high-frequency attenuation) correlates with the input through trivial shared structure rather than through genuine reconstruction. The 0.3 threshold proposed initially as a working heuristic was therefore found in retrospect to be insufficient as a discriminator, and listening evaluation in Chapter 5 became the primary perceptual evaluator for canonical-checkpoint selection.

Thirdly, the $[0.05, 0.3]$ amplitude band used as a sanity check is an engineering heuristic chosen to flag clipping and silent-output failures; it is not derived from a perceptual model of loudness.

Fourthly, no formal perceptual evaluation (such as MUSHRA or MOS testing) was conducted, due to the exploratory and iterative nature of the experiments and because many intermediate models failed basic reconstruction criteria before formal perceptual comparison would have been meaningful. Listening evaluation was conducted informally by the author at each checkpoint.

Together, these protocols allow each training iteration in Chapter 5 to be evaluated against consistent quantitative criteria, while remaining transparent about what those criteria do and do not measure.

3.5 Methodological Assumptions and Limitations

The methodology described in this chapter rests on a number of working assumptions whose limits delimit the scope of the claims that Chapter 5 can support. These are stated up front rather than deferred to the conclusions, complementing the evaluation caveats already given in §3.4.

Cross-modal generalisation assumption. The system is trained as an instrument-domain autoencoder and is required to generalise to a categorically different excitation signal (voice) at inference time. There is no theoretical guarantee that an autoencoder trained only on guitar or drums will represent vocal input meaningfully in latent space. The project proceeds under the working assumption that learned latent representations carry sufficient spectral-temporal generality to admit at least partial cross-modal mapping. The failure of this assumption on the small open corpora used here is itself one of the documented findings (§5.2, §5.3).

Dataset scale. The guitar corpus (~9 h raw, ~16 h after RAVE’s overlap segmentation) is small by contemporary generative-audio standards; several published RAVE demonstrations use corpora an order of magnitude larger. Whether the failure modes observed in this work persist at greater dataset scale is not investigated, and replication on larger corpora is identified as future work (§6.4).

Dataset heterogeneity. The guitar corpus mixes microphone-captured acoustic, pickup-captured acoustic, and direct-input electric recordings. The noise-floor analysis of §5.5 quantifies a 35.9 dB spread between source conditions. The work does not control for this heterogeneity by restricting training to a

single capture path, and the resulting latent representation almost certainly encodes recording method as a salient feature.

Single-evaluator perceptual judgments. As stated at the head of §3.4, listening evaluation was conducted informally by the author. Claims of “working” or “recognisable” reconstruction throughout Chapter 5 reflect a single trained ear rather than statistically validated perceptual quality. Formal perceptual evaluation with multiple listeners is identified as future work (§6.4).

Analytic rather than calibrated latency. The end-to-end latency reported in Table 3.1 is an analytic sum of component latencies (block size, model forward, reported driver buffers), not a calibrated loopback measurement. The estimate is internally consistent with the pipeline design but should not be taken as a precise empirical figure; calibrated loopback measurement is identified as future work.

Single workstation, single trial per configuration. Each configuration was trained once. Stochastic variation across seeds was not characterised. Where this matters — particularly for adversarial training, which is known to be seed-sensitive — the observed outcomes should be read as representative of the configuration under one realisation rather than as a population statistic.

The implementation details that operationalise this methodology — software environment, compatibility patches, training scripts, and Pure Data deployment patches — are described in the following chapter.

4. Implementation

This chapter describes how the methodology of Chapter 3 was operationalised in software and deployed for real-time use. Implementation choices are linked explicitly to the project’s research objectives: sub-10 ms latency, encoder responsiveness, and reproducible training.

4.1 Environment and Compatibility

The BRAVE resynthesis pipeline (Caillon & Esling, 2021) was deployed on a single workstation. The complete software and hardware stack is summarised in Table 4.1.

Table 4.1: Hardware and software environment used for all training and deployment in this project.

Component	Version
Operating system	Windows 11 (build 26200+)
Python	3.14.3
PyTorch	2.11.0+cu128
CUDA driver	13.2
GPU	NVIDIA RTX 5080 (16 GB VRAM)
acids-rave	2.3.1
pytorch-lightning	2.6.1
scipy	≥ 1.14
Audio tooling	librosa, soundfile, ffmpeg
Real-time host	Pure Data 0.55 with the <code>nn~</code> external

During training, the RTX 5080 supported an observed throughput of 9–12 optimisation steps per second. Each full 3,000,000-step run therefore required 90–120 wall-clock hours, which made GPU capacity the primary constraint on iteration speed.

A notable implementation challenge involved upstream library drift in acids-rave 2.3.1, which was originally authored against earlier SciPy and PyTorch Lightning ecosystems. Four compatibility patches were applied to the library source, grouped into three categories: two SciPy API updates (a relocated `scipy.signal.kaiser` import and a type-coerced `kaiserord` argument), one SciPy parameter rename (firwin’s `nyq` argument to `fs`), and one PyTorch Lightning 2.x API rename (`validation_epoch_end` to `on_validation_epoch_end`). Together they total fewer than ten lines of changed code. The exact diffs are reproduced in Appendix-C and in the public repository to enable independent verification of the environment.

4.2 Training Pipeline

Training was managed through the `.gin` configuration system, which allowed modular and hierarchical hyperparameter overrides. Rather than treating each configuration as an isolated artefact, the project produced a chain of configurations in which each step responds to a failure mode observed in the previous iteration. This progression is summarised in Table 4.2.

Table 4.2: Configuration evolution across guitar iterations and the two drums iterations. Each row identifies the main change introduced and the failure mode it addressed.

Iteration	Configuration	Main change introduced	Motivation
guitar_v1	unmodified c128_r10.gin	— (baseline)	first run; revealed posterior collapse
guitar_v2	c128_r10_beta_fixed.gin	Log-space β warmup ($1 \times 10^{-4} \rightarrow 0.1$)	Eliminate posterior collapse
guitar_v3, guitar_v4	c16_r10_beta_fixed.gin	LATENT_SIZE reduced 128 $\rightarrow$ 16	Address 0.005 nats-per-dimension utilisation
guitar_v5	c16_r10_v5_balanced.gin	Discriminator capacity halved and shallower	Counter discriminator dominance observed in v4
guitar_v6	guitar_v6_overrides.gin (on official v2.gin + causal.gin)	Adopt official architectural baseline; extend Phase 1 to 1.5 M steps	Replace manual architectural tuning with a community-validated configuration
drums_v1	c16_r10_beta_fixed.gin	Same custom config applied to Groove drums corpus	First drums case study under the c16_r10 family of configs; achieved partial transient response (see §5.2)
drums_v2	drums_v2_overrides.gin	Apply the guitar_v6 configuration template to the Groove corpus	Extend the architectural baseline to the drums case study (training initiated; terminated before completion — discussed in §5.2)

To manage interruptions during multi-day runs, each iteration is launched through a Windows .bat script. The script invokes PowerShell to find the most recent checkpoint matching the run name; if one is found, training resumes via the `--ckpt` flag, otherwise it starts fresh. This made the pipeline safely interruptible and was used multiple times after both planned and unplanned system restarts.

The checkpointing strategy used `--val_every 10000`, saving model state every ten thousand optimisation steps. Three redundant levels are maintained: the RAVE-native `best.ckpt` (selected by lowest validation reconstruction loss), epoch-tagged files (e.g. `epoch=2511.ckpt`), and fixed snapshots every 500,000 steps. As discussed in §3.2, this redundancy proved important for guitar_v6, where reconstruction quality peaked around epoch 1935 during Phase 2 and degraded slightly toward the final epoch. The canonical model `guitar_v6_best.ts` is therefore the mid-trajectory Phase 2 checkpoint rather than the final one.

Reproducibility. All configuration files, preprocessing scripts, and evaluation utilities are version-controlled in the project repository and will be released alongside the thesis. The repository is structured so that the documented iterations can be independently regenerated through the same configurations and preprocessing pipeline.

4.3 Real-Time Deployment

Model deployment followed the standard acids-rave export workflow. The command `rave export --run <RUN_DIR> --name <NAME>` compiles the trained PyTorch graph into a TorchScript .ts file that bundles the learned weights with the cached convolutional state required for incremental inference.

The inference environment uses the `nn~` Pure Data external, chosen for two reasons that connect directly to the project’s research objectives. First, `nn~` provides a stable, community-maintained deployment target that allows the sub-10 ms latency budget defined in §3.1 to be achieved without the development overhead of a custom C++ audio engine. Second, Pure Data integrates standard DSP blocks (gain, limiters, filtering) natively, which makes it straightforward to compensate for the trained model’s idiosyncrasies

without inserting additional inference stages on the audio path.

The deployed signal chain follows a linear path:

$$\text{adc} \sim 1 \rightarrow \text{nn} \sim \langle \text{model} \rangle . \text{ts forward} \rightarrow * \sim \langle \text{gain} \rangle \rightarrow \text{dac} \sim$$

A reference block diagram is provided in Figure 3.1. The microphone input is mapped to the model’s forward method, which receives a tensor of shape (1, 1, 64) for each audio block.

Sample-rate compatibility. A reproducibility consideration identified during deployment testing involved the Pure Data audio engine’s sample-rate default. A default rate of 48,000 Hz was observed to produce silent or unusable output because it mismatched the 44,100 Hz used during training and LMDB construction. The requirement to set the sample rate explicitly to 44,100 Hz in `Media → Audio settings` is therefore documented as a mandatory deployment step.

Gain compensation. BRAVE decoder output amplitudes typically fall within the [0.005, 0.05] range. As a consequence, the unscaled output of `nn~` is generally below practical listening level when connected directly to `dac~`. Fixed multipliers were tuned empirically against the measured output amplitude of each model: $\times 50$ for lower-energy outputs such as `drums_v1`, and $\times 20$ to $\times 30$ for the higher-energy `guitar_v6`. The compensation was implemented as a single `* ~` multiplier on the audio path so that audible listening levels could be reached without introducing additional DSP stages that could erode the latency budget.

4.4 Evaluation Tooling

Several diagnostic tools were implemented to quantify model behaviour beyond informal listening. A custom Python script, `scripts/analyze_noise_floor.py`, computes the 5th percentile of 100 ms RMS windows for each audio file in the guitar corpus and aggregates the results per source. Running this script across all 919 source recordings produced the per-source noise-floor distribution discussed in §5.5, including the 35.9 dB spread between electric DI and microphone-captured acoustic sources.

The responsiveness of each candidate checkpoint was measured by a separate synthetic-input test. The script passes five spectrally distinct signals — silence, three sine tones (220, 440 and 880 Hz), a frequency sweep, and white Gaussian noise — through the model and computes pairwise mean absolute differences in latent vectors and audio outputs. These differences are compared against the health thresholds defined in §3.4. The thresholds were selected empirically after comparing responsive and collapsed checkpoints across early exploratory runs. Collapsed models produced pairwise latent differences below 0.2 and pairwise output differences below 0.005, whereas functionally responsive checkpoints consistently exceeded 0.5 and 0.01 respectively. The same script flagged the early collapsed iterations — `guitar_v1` (output difference = 0.006) and `guitar_v2` (output difference = 0.003) — as well as the partial-response state of `guitar_v3` (output difference = 0.042).

Training health was monitored continuously through TensorBoard (`tensorboard --logdir runs/`). Critical scalars, including KL divergence and the discriminator logit gap between `pred_real` and `pred_fake`, were tracked throughout training. The same instrumentation allowed early identification of discriminator dominance in `guitar_v4` (gap = 5.65) and of the inverse failure in `guitar_v5`, where both logits became negative and the gap collapsed to approximately 0.8.

Combined, these tools turned the implementation from a trial-and-error process of architectural experimentation into a reproducible and diagnosable training pipeline that meets the latency and responsiveness requirements for live musical interaction (Wessel & Wright, 2002). The diagnostics and checkpoint analyses that came out of this process became the foundation of the comparative evaluation presented in Chapter 5.

5. Results and Evaluation

This chapter assesses the strengths and weaknesses of the BRAVE-based resynthesis pipeline when used in real-time situations. The study concentrates on the stability of latent representations, the behaviour of adversarial training, and the ability to generalise out-of-distribution (OOD) across the guitar and drum resynthesis tasks. Results are examined both quantitatively and perceptually in an attempt to identify the main obstacles to efficient streaming neural audio resynthesis.

The investigation is divided into two main case studies: a six-iteration development arc for guitar resynthesis and an additional evaluation of drum-kit resynthesis. As described in Chapter 3, pipeline performance was measured against diagnostic criteria for encoder separation (pairwise latent difference greater than 0.5), decoder sensitivity (pairwise output difference greater than 0.01), and energy conservation (output level within $[0.05, 0.3]$). In-distribution reconstruction was checked by a STFT magnitude correlation rule of greater than 0.3. All live experiments were conducted within a Pure Data environment using the `nn~` external with an estimated end-to-end delay of approximately 7 ms.

5.1 Guitar: Six-Iteration Arc

The guitar example focused on upgrading the training procedure step by step, starting from the baseline settings and progressing to the official RAVE v2 architecture. Each step was instrumented through the diagnostic Python scripts described in Chapter 4.

5.1.1 Iterations v1–v2: Latent Space Failures

The first version (`guitar_v1`) was based on the `c128_r10.gin` file without any changes. This run was affected by posterior collapse, in which the latent space becomes unusable because the Kullback–Leibler (KL) component is too strong from the beginning and pushes the solution toward the prior (Bowman et al., 2016). TensorBoard diagnostics indicated that the KL regularisation term decreased from approximately 0.83 to 0.25 over the duration of training (Figure 5.1). Although the encoder could distinguish between inputs (encoder difference = 1.25), the decoder effectively discarded the latent information, producing a forward difference of 0.006, well below the 0.01 health threshold. A log-space β warmup was therefore concluded to be necessary for the model to first learn significant reconstructions before being regularised.

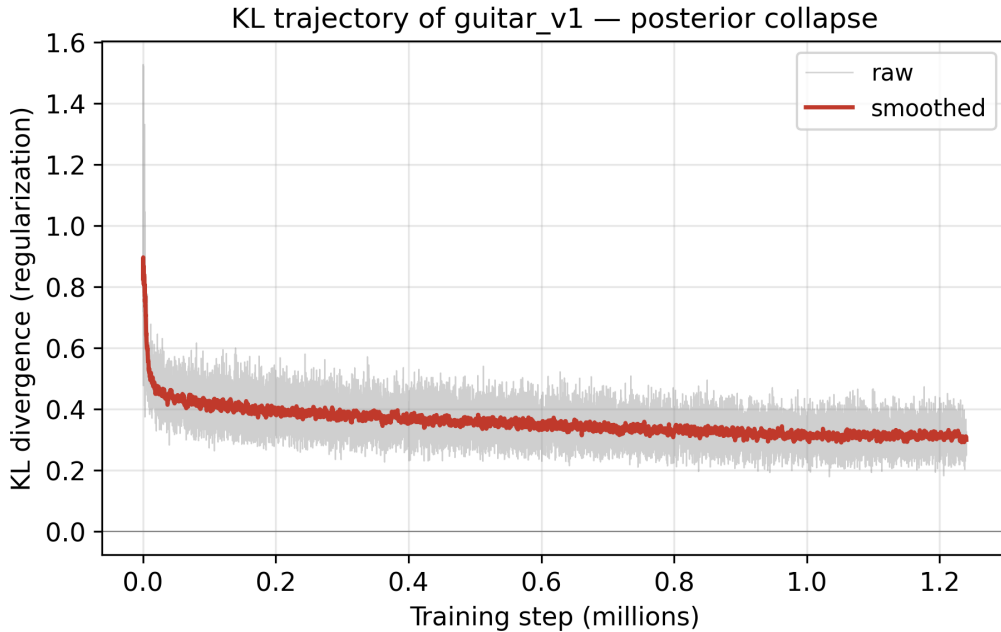


Figure 5.1: KL divergence trajectory for guitar_v1 during Phase 1 training. The KL term falls from approximately 0.83 at step zero to 0.25 by step 1.24 M. A decreasing KL coupled with a decoder forward difference of 0.006 (below the 0.01 health threshold) constitutes the diagnostic signature of posterior collapse (mode 1 of the failure taxonomy, §5.4).

In the next run (guitar_v2) the β warmup was applied and posterior collapse was no longer a problem. The model, however, did suffer a partial collapse because the latent space was too large. With LATENT_SIZE of 128, the model achieved a utilisation of only approximately 0.005 nats per dimension. The diagnostics revealed a small encoder difference (0.79) and a decoder that was not reacting at all (forward difference = 0.003). The output amplitude of 0.012 was roughly 60 times lower than the input signal. This implied that the 128-dimensional latent vector was too large for the ~ 5.4 -hour corpus available at that stage, and a reduction to 16 dimensions was therefore necessary.

5.1.2 Iterations v3–v5: Adversarial Instabilities

The third and fourth iterations explored how dataset composition affected the model with the latent size reduced to 16. guitar_v3, trained on 5.79 hours of mixed data, exhibited a partial response with forward differences peaking at 0.042. However, the model experienced Phase 2 generator collapse, meaning that the custom discriminator dominated the adversarial game.

guitar_v4 attempted to remedy this by using a cleaner, microphone-only dataset (3.05 h compared to v3’s 5.79 h). Counter-intuitively, the reduction in data volume worsened the failure, leading to adversarial mode collapse. TensorBoard scalars revealed clear discriminator dominance: `pred_real` reached 3.34 while `pred_fake` fell to -2.31 , producing a logit gap of 5.65 (Figure 5.2). Under these conditions the decoder produced a silence amplitude larger than the signal amplitude.

A focused experiment aimed at balancing the adversarial game was conducted in guitar_v5 by halving the discriminator’s capacity. This produced the inverse failure: discriminator collapse. The `pred_real` logit fell to -0.90 , indicating that the discriminator was assigning negative scores to real audio, and the gap narrowed to 0.80. These findings, summarised in Table 5.1, indicated that no simple parameter scaling of the custom architecture could resolve the dominance/collapse axis.

Figure 5.2. Adversarial dynamics across iterations

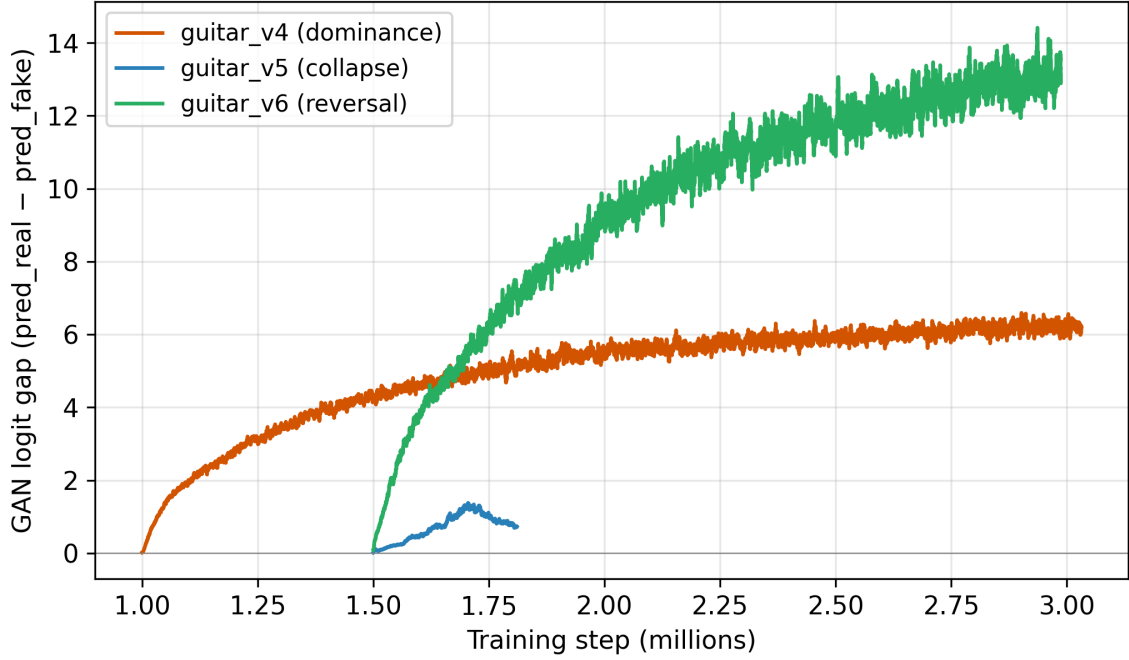


Figure 5.2: Adversarial dynamics across the three Phase-2 iterations: guitar_v4 exhibits a widening GAN logit gap (peaks at 5.65, discriminator dominance); guitar_v5 collapses to a narrow gap of approximately 0.80 (discriminator collapse); guitar_v6 is the first checkpoint where the gap reverses direction during Phase 2, indicating an approximate adversarial equilibrium.

Table 5.1: Summary of failure modes across the guitar development iterations.

Iter.	Configuration	Failure mode	Diagnostic signature
v1	Baseline c128_r10.gin	Posterior collapse	KL falls $0.83 \rightarrow 0.25$; forward diff 0.006
v2	β warmup applied	Latent underutilisation	KL/dim ≈ 0.005 nats; forward diff 0.003
v3	LATENT_SIZE $\rightarrow 16$	Phase 2 collapse	Forward diff peaks at 0.042 then degrades
v4	Mic-only data (3.05 h)	Adversarial mode collapse	Gap = 5.65; silence amp. > signal amp.
v5	Halved discriminator	Discriminator collapse	pred_real = -0.90; gap = 0.80
v6	Official RAVE v2 base	Partial in-distribution reconstruction	STFT corr. calibrated in Table 5.2; GAN-gap reversal

5.1.3 Iteration v6: Partial In-Distribution Reconstruction

The final iteration (guitar_v6) was based on the RAVE v2 official architecture and causal.gin overrides. In Phase 2 the GAN logit gap was observed to reverse for the first time in the series, suggesting that an adversarial equilibrium had been reached. Listening tests indicated perceptually recognisable reconstruction of in-distribution guitar audio at epoch 1935 (mid-Phase-2), and this checkpoint was therefore selected as canonical. The final epoch (2511) showed slightly degraded performance under the same listening criteria. Under the operational diagnostic of §3.4, output amplitude rose from the near-silent levels of the collapsed checkpoints v1–v3 (output RMS ≈ 0.003 – 0.005) to RMS ≈ 0.024 on the canonical guitar_v6 checkpoint and 0.041 on the Phase 1 reconstruction-only state, which together suggest a meaningful recovery of decoder responsiveness.

A quantitative reconstruction test on the canonical checkpoint, using eight GuitarSet solo-microphone samples of five seconds each (STFT parameters: $n_{\text{fft}} = 2048$, $\text{hop} = 512$), yielded a mean STFT magnitude correlation of 0.16 (0.17 with time-shift alignment). This value falls below the 0.3 heuristic proposed initially in §3.4. To understand whether this gap reflects a limitation of the model or of the metric, the same evaluation was applied to every preceding checkpoint in the iteration sequence; the results are reported in §5.1.4. Anticipating the finding of that calibration: the STFT magnitude correlation does not differentiate working from collapsed checkpoints in the expected direction, and the operational evaluation of guitar_v6 as a partially-reconstructing in-distribution autoencoder therefore rests on the combined evidence of (i) listening tests, (ii) output amplitude recovery, and (iii) Phase 2 GAN gap reversal.

Figure 5.4. In-distribution reconstruction example (00_Jazz3-137-Eb_solo_mic.wav)

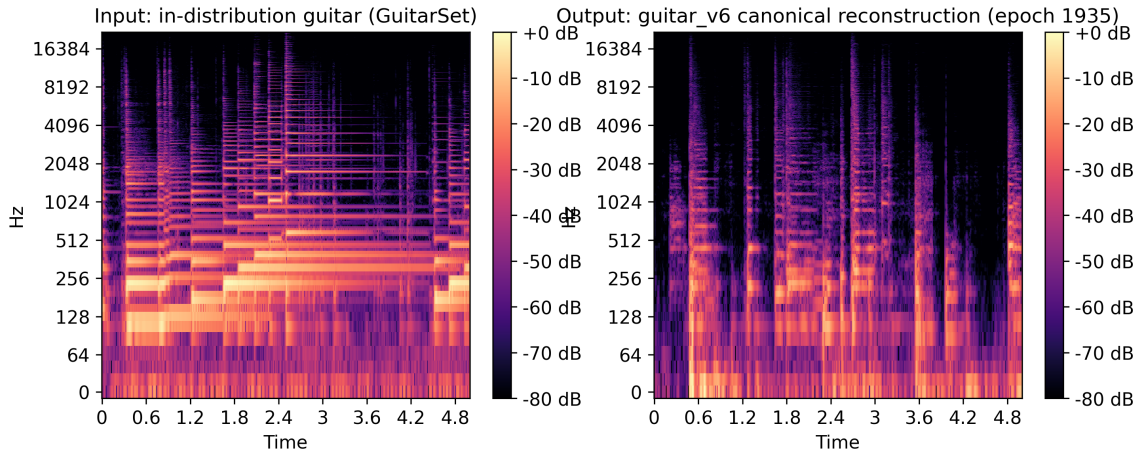


Figure 5.3: STFT magnitude spectrograms (log scale) for an in-distribution GuitarSet sample: input (left) and the canonical guitar_v6 reconstruction (right). The reconstructed spectrogram preserves the harmonic structure and onset patterns of the input although it lacks fine spectral detail above approximately 4 kHz.

The same in-distribution success did not transfer to voice input, which is out-of-distribution (OOD) for the trained model. The encoder was capable of differentiating the vocal signals — latent differences between distinct vocal inputs ranged from 1.27 to 6.89 — but the decoder produced a practically uniform, low-energy output. This suggests that the encoder can map voice into latent space, but the resulting trajectories lie outside the trained guitar manifold, and the decoder lacks the generative coverage required to render them.

5.1.4 Calibration of the STFT-Magnitude-Correlation Diagnostic

To assess whether the STFT magnitude correlation criterion of §3.4 robustly discriminates working autoencoders from collapsed ones, the same in-distribution reconstruction test was applied to every checkpoint in the iteration sequence. Eight GuitarSet solo-microphone samples of five seconds each were evaluated against each checkpoint with identical STFT parameters ($n_{\text{fft}} = 2048$, $\text{hop} = 512$). Results are summarised in Table 5.2.

Table 5.2: Cross-checkpoint calibration of the STFT-magnitude-correlation diagnostic. All values are means across $N = 8$ GuitarSet samples. Output RMS reflects the energy of the model output relative to the input (mean input RMS ≈ 0.065).

Checkpoint	Diagnostic state	Output RMS	STFT (raw)	STFT (aligned)
guitar_v1	Posterior collapse	0.005	0.16	0.16
guitar_v2	Latent underutilisation	0.003	0.24	0.25
guitar_v3	Phase 2 generator collapse	0.004	0.24	0.26
guitar_v4	Adversarial mode collapse	0.021	0.25	0.26
guitar_v5	Discriminator collapse	0.024	0.26	0.28
guitar_v6 Phase 1 (ep. 1499)	Reconstruction-only	0.041	0.19	0.21
guitar_v6 best (ep. 1935)	Partial reconstruction	0.024	0.16	0.17
	(canonical)			
guitar_v6 final (ep. 2511)	Working, slightly degraded	0.024	0.14	0.17

Two observations follow from Table 5.2. First, no checkpoint reaches the 0.3 heuristic that was proposed initially in §3.4 as the threshold for “perceptually acceptable” reconstruction; the maximum observed value across all checkpoints is 0.28 (time-aligned, guitar_v5). Second, and more revealing, the STFT magnitude correlation does not differentiate between working autoencoders and known-collapsed checkpoints in the expected direction: collapsed checkpoints v2–v5 score equal to or slightly higher than the guitar_v6 canonical checkpoint.

The reason is structural rather than incidental. Collapsed checkpoints produce near-silent outputs (RMS ≈ 0.003 – 0.024 versus input RMS ≈ 0.065); the STFT magnitudes of such outputs are uniformly small but nonetheless share the general spectral envelope of any natural audio source (low-frequency dominance, high-frequency attenuation). The flattened Pearson correlation between this trivial envelope and the input’s STFT therefore captures shared spectral shape rather than genuine reconstruction. Output RMS, by contrast, does discriminate: collapsed checkpoints produce outputs an order of magnitude quieter than guitar_v6’s Phase 1 reconstruction-only state (guitar_v6 Phase 1, RMS = 0.041).

This finding constitutes a calibration limitation of the diagnostic methodology defined in §3.4. The STFT magnitude correlation criterion was empirically derived from preliminary comparisons but was not stress-tested against the full iteration sequence at the time of definition. The full calibration in Table 5.2 reveals that the metric was insufficient as a sole discriminator; output amplitude and informal listening evaluation were therefore retained as the operational evaluators for canonical-checkpoint selection (§5.1.3). The threshold of 0.3 should be understood as a working initial proposal rather than a validated criterion; future replication studies on this diagnostic framework should re-derive the threshold using outputs from both working and collapsed reference models in the target setup.

5.2 Drums

The drum-kit evaluation aimed to apply the BRAVE streaming-causal recipe of Caspe et al. (2025) to the 10.86-hour Groove corpus.

The initial drum iteration (drums_v1), using the `c16_r10_beta_fixed.gin` configuration, resulted in a forward difference of 0.0045. This was below the 0.01 threshold defined for tonal responsiveness. Initial informal listening tests suggested that voice consonants and beatbox transients gave rise to recognisable drum-like timbres in Pure Data, although subsequent re-listening at a later date with the same test setup did not reproduce a perceptually compelling voice-to-drum mapping. The numerical diagnostic indicates that a “working voice-to-drums” claim cannot be substantiated on spectral evidence alone.

The second iteration (drums_v2) applied the configuration template established in guitar_v6 (the official v2.gin architectural baseline with the project’s causal overrides) to the Groove corpus and was trained for approximately 95 hours (≈ 2.28 M steps). Phase 1 concluded with healthy validation scores

($\text{val} \approx 4.1$, $\text{mb_spec} \approx 3.7$). However, Phase 2 exhibited a monotonic widening of the GAN gap from 9.8 to 20.3 over 800,000 steps (Figure 5.4), while the validation loss plateaued around 7.5. Training was terminated before the scheduled time as discriminator dominance reproduced despite the `update_discriminator_every` safeguard provided by the v2 architecture. This indicates that the `guitar_v6` configuration template did not transfer cleanly from the guitar dataset to the drum dataset under these specific training constraints.

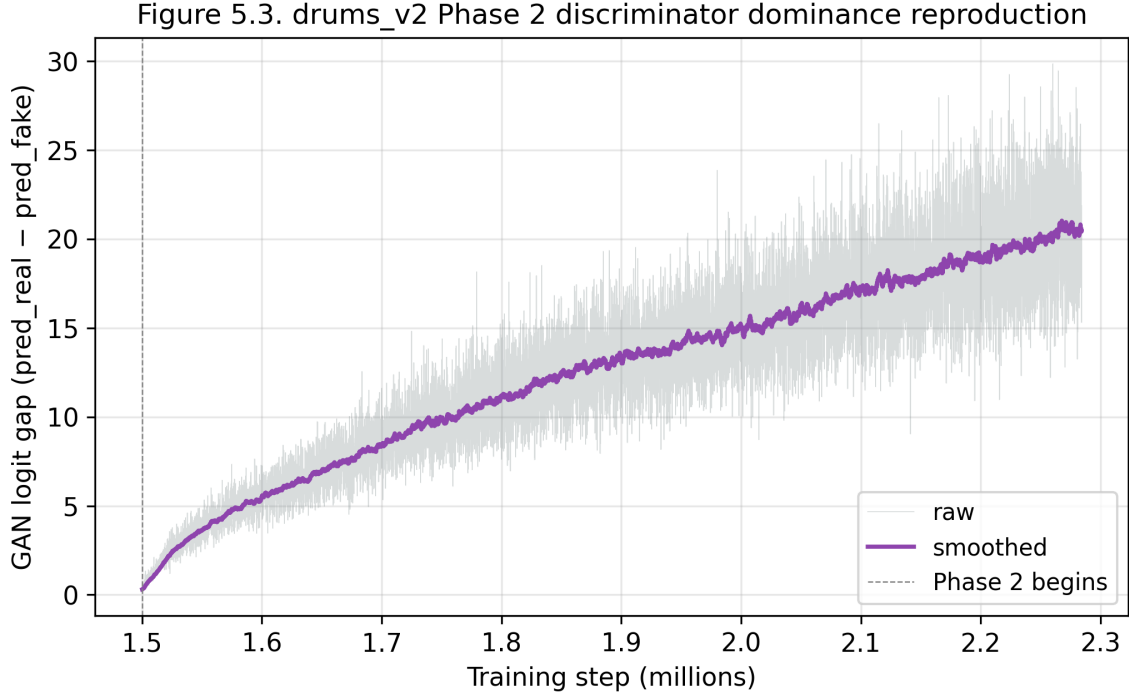


Figure 5.4: drums_v2 GAN logit gap over the duration of training. Phase 1 (steps 0–1.5 M) maintains a near-zero adversarial gap. From Phase 2 onset onward, the gap widens monotonically from approximately 9.8 to over 20, reproducing the discriminator-dominance pattern previously observed in guitar_v3/v4 despite the use of `update_discriminator_every=4`.

5.3 External Baseline: IRCAM Percussion

To determine whether the OOD limitations observed in iterations v6 and drums_v1 were inherent to the BRAVE architecture, a comparative diagnostic was performed using the official ACIDS-IRCAM percussion model (71 MB TorchScript) (ACIDS-IRCAM, 2024). When subjected to the same synthetic-input test described in §3.4, the IRCAM model yielded a forward difference (silence vs. noise) of 1.75. This indicates that the official model is more responsive than the drums_v1 iteration (forward difference 0.0045) by more than two orders of magnitude under this diagnostic.

The IRCAM model respected silence (0.001 output for silent input) and, in informal Pure Data listening tests with voice input, produced audibly modulated percussive output. This indicates that a well-trained BRAVE model with higher-quality or more voluminous data can overcome the manifold-mapping bottleneck. This comparison supports the conclusion that the performance-limiting factors of the present models are training recipe and data scale rather than fundamental architectural constraints of BRAVE.

5.3.1 Candidate Explanations for the IRCAM Responsiveness Gap

The IRCAM percussion baseline being more responsive than drums_v1 by more than two orders of magnitude on the same diagnostic raises the question of what specifically in IRCAM’s training recipe accounts

for the gap. The official model was not accompanied by a training-protocol publication at the time of writing, so the candidates below are framed as informed hypotheses grounded in the broader RAVE/BRAVE literature rather than as established findings. Each is identified as a candidate for empirical follow-up in §6.4.

Hypothesis 1 — Training data scale. The published RAVE training compendium recommends a minimum of three hours of in-distribution audio and suggests that significantly larger corpora (tens of hours upward) produce more robust models. Although the present drums corpus (Groove MIDI Dataset, ≈ 10.86 h) satisfies the recommended minimum, it is plausible that the IRCAM percussion model was trained on a larger corpus collected from multiple percussion sources. Larger datasets are known empirically to mitigate the discriminator-dominance pattern that terminated drums_v2 (§5.2).

Hypothesis 2 — Dataset homogeneity. Our consolidated guitar corpus exhibited a 35.9 dB noise-floor spread across recording paths (§5.5), which the encoder is well positioned to encode as a salient latent feature at the expense of timbral discrimination. Although the drums corpus did not undergo the same per-source analysis, the Groove MIDI Dataset combines multiple drummers and kit configurations. Curated single-source corpora — which IRCAM-quality pretrained models are typically trained on — avoid this heterogeneity penalty.

Hypothesis 3 — Architectural variant (RAVE v3 with AdaIN). Recent ACIDS-IRCAM releases have incorporated the RAVE v3 architecture, which adds Snake activations, a Descript-style discriminator, and Adaptive Instance Normalisation (AdaIN) for explicit cross-domain style transfer. AdaIN in particular is designed to bridge timbre gaps between source and target domains in a way that v1 and v2 architectures are not. If the IRCAM percussion checkpoint was produced with a v3 configuration — which we cannot determine without access to its training metadata — the responsiveness to OOD voice input would not be surprising. The 32 GB VRAM requirement of v3 prevented us from testing this hypothesis directly on the present 16 GB GPU.

Hypothesis 4 — Training duration and Phase 2 schedule. Our iterations targeted approximately 3 M total steps with a Phase 1 of 1.5 M for v6 and drums_v2. Several community-reported recipes extend Phase 2 beyond 3 M steps before declaring training complete, particularly for percussion datasets where transient quality is dominated by adversarial fine-tuning rather than spectral reconstruction. The drums_v2 trajectory was terminated at 2.28 M (mid-Phase-2) for resource reasons; it is open whether further training would have stabilised or aggravated the dominance pattern.

Hypothesis 5 — Augmentation. The acids-rave 2.3.1 training pipeline exposes `--augment gain` `--augment mute` flags, but in our setup these are non-operational: `compress` and `RandomCompress` depend on `torchaudio.sox_effects` which is removed in current torchaudio releases, and inspection of the upstream `RandomGain` transform shows a bug whereby the gain-adjusted signal `x_amp` is computed but the original `x` is returned. Reference IRCAM training pipelines may use augmentation either through patched versions of the same library or through external preprocessing; this would teach the encoder amplitude- and silence-invariance that our trained encoder lacks.

Disentangling these candidates would require either access to the IRCAM training metadata or a controlled replication study varying one factor at a time on our infrastructure. Both are identified as priorities for the future work outlined in §6.4.

5.4 Failure-Mode Taxonomy

The outcomes of the iterative training campaign can be summarised into five chief failure modes relevant to streaming VAE resynthesis. These modes, discovered through the Python diagnostics described in Chapter 4, serve as guides to the stabilisation of similar real-time models.

1. **Posterior collapse (v1).** Identifiable by a falling KL trajectory and a decoder that ignores the latent space. Caused by applying KL pressure before reconstruction has been learned.
2. **Latent underutilisation (v2).** Identifiable by very low nats-per-dimension (≈ 0.005) and a near-silent decoder. Occurs when the bottleneck capacity exceeds the dataset’s information density.
3. **Discriminator dominance / generator mode collapse (v3, v4, drums_v2).** Marked by a widening discriminator logit gap (typically > 5) and, in the worst case, silence amplitude exceeding signal amplitude. Triggered when the default custom discriminator out-paces the generator during Phase 2.
4. **Discriminator collapse (v5).** The inverse failure — heuristic discriminator weakening pushed `pred_real` to negative values and narrowed the logit gap to 0.80. Confirmed that no simple capacity-scaling resolves the dominance/collapse axis.
5. **OOD decoder behaviour (v6, drums_v1).** A structural failure where the encoder differentiates non-instrument input (e.g. voice), but the decoder fails to render it because the resulting latent coordinates reside outside the trained manifold.

As the IRCAM baseline comparison (§5.3, §5.3.1) confirms, these should be regarded as training-recipe failures rather than architectural limitations of BRAVE. Addressing them in future work will require more homogeneous datasets and possibly explicit pitch-conditioning to close the cross-modal gap.

5.5 Noise-Floor Heterogeneity in the Guitar Corpus

Chapter 3, §3.3 introduced the heterogeneity concern that arises from combining recordings captured through different paths into a single training corpus. To characterise that heterogeneity quantitatively, the noise-floor methodology defined in §3.3 was applied to every source recording of the consolidated guitar corpus (919 files across six recording paths). For each file, the Root Mean Square energy was computed in non-overlapping 100 ms windows; the 5th percentile of these RMS values was taken as the noise-floor estimate and converted to dBFS. Per-source aggregates are reported in Table 5.3 and visualised in Figure 5.5. The analysis was performed on the source recordings; the 52 Guitar-TECHS files measured here were reduced to 32 in the final training corpus of Table 3.2 (899 files in total) after 20 stereo files were removed during the mono-conversion stage, so the two totals differ only by that filtering step.

Table 5.3: Per-source noise-floor distribution across the source recordings of the consolidated guitar corpus (919 files; see text for the relationship to the 899-file training corpus of Table 3.2). The 5th-percentile RMS in 100 ms windows estimates the noise floor of each recording.

Source	<i>n</i> files	Recording method	Noise floor (dBFS, median)	IQR (dB)
GuitarSet	360	Mic-captured acoustic	−46.5	10.4
Guitar-TECHS	52	Electric direct input	−67.2	28.1
IDMT acoustic_mic	128	Mic-captured acoustic	−57.8	7.1
IDMT acoustic_pickup	128	Pickup capture	−53.3	9.4
IDMT Career SG	123	Electric direct input	− 82.4	7.2
IDMT Ibanez 2820	128	Electric direct input	−71.5	11.3

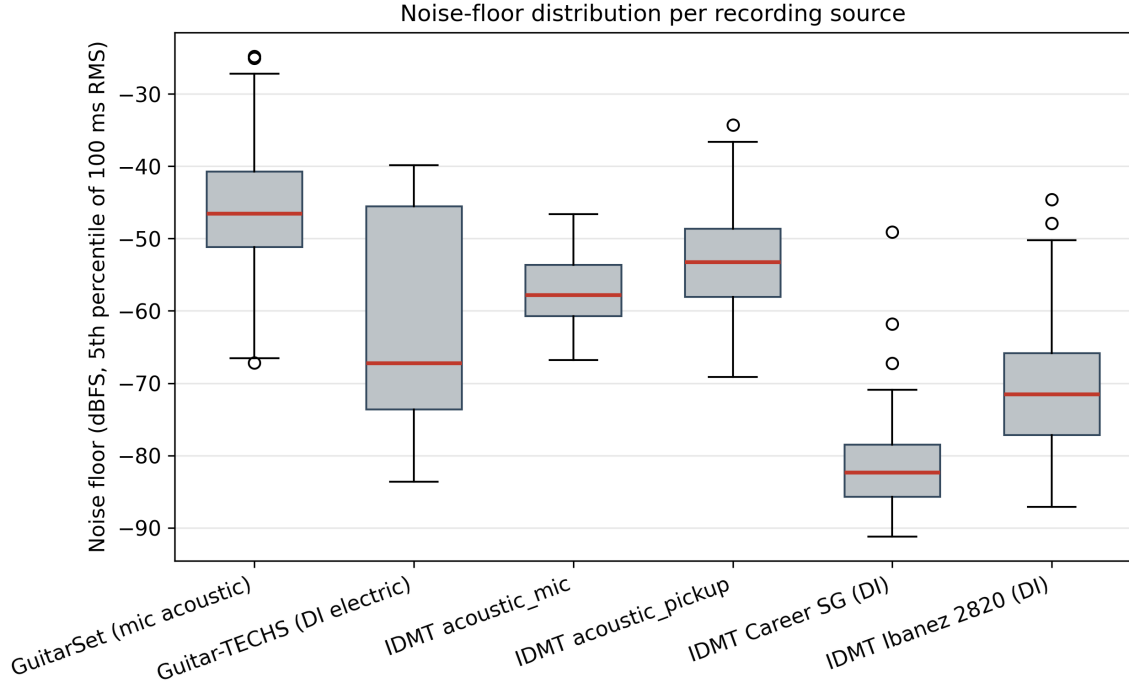


Figure 5.5: Per-source noise-floor distribution across the source recordings of the consolidated guitar corpus (box-plots; $n = 360, 52, 128, 128, 123, 128$ files respectively). Median noise floors range from -82.4 dBFS (IDMT Career SG DI) to -46.5 dBFS (GuitarSet mic-captured acoustic), a 35.9 dB spread that exceeds typical within-performance dynamic variation.

The spread between the cleanest source (IDMT Career SG direct input, median -82.4 dBFS) and the loudest source (GuitarSet mic-captured acoustic, median -46.5 dBFS) is 35.9 dB. To contextualise that figure, the typical dynamic variation within a single guitar performance is approximately 20 – 30 dB between *pi-anissimo* and *fortissimo* passages, so the inter-source noise-floor spread exceeds the within-performance dynamic range by an order of magnitude.

A consequence anticipated in §3.3 is that the encoder may treat the recording method as a salient latent feature, allocating capacity to discriminate among capture paths rather than among timbral characteristics. The present study did not directly test this hypothesis through a controlled latent-clustering experiment; this is identified as a candidate analysis for future work (§6.4). However, the magnitude of the per-source spread reported here provides quantitative grounding for the limitation acknowledged throughout the thesis: the combined corpus, although it satisfies RAVE’s recommended duration window (Caillon & Esling, 2021), is heterogeneous on a dimension that the model is well positioned to encode.

Two practical implications follow. First, the iteration sequence’s reliance on the combined corpus from v3 onwards (after the smaller v1/v2 single-source corpus failed to support a 128-dimensional latent) made dataset homogeneity a secondary priority to dataset scale, a trade-off discussed in §3.3 and revisited in §6.4. Second, the contrast with the IRCAM percussion baseline (§5.3) — trained on a presumably more curated corpus — supports the conclusion that training-recipe choices including corpus selection rather than BRAVE architecture per se account for the responsiveness gap.

6. Conclusions and Future Work

6.1 Summary of Findings

This thesis explored the feasibility of the streaming-causal BRAVE architecture for voice-driven instrument resynthesis with less than 10 ms latency when trained on heterogeneous, limited-scale datasets. The study went through six versions of the guitar model (v1–v6) and two versions of drums (drums_v1, drums_v2). It was found that guitar_v6, which used the official v2.gin and causal.gin configurations along with a smaller latent size of 16 and a longer Phase 1 of 1.5 M steps, was the first iteration to partially reconstruct in-distribution input — preserving harmonic structure and onsets, though with limited high-frequency detail (Chapter 5). On the other hand, the drums_v2 version was stopped prematurely as the Phase 2 discriminator dominance appeared to re-emerge even though an `update_discriminator_every=4` schedule had been implemented.

It was concluded that voice remained an out-of-distribution (OOD) input across all the guitar checkpoints — the main goal of the cross-modal resynthesis (musical instruments from voice). The encoder seemed to distinguish voice inputs, with differences in latents observed ranging from 1.27 to 6.89, but the decoder output was nearly the same in all cases, signalling a structural failure in associating voice latents with the trained guitar manifold. Moreover, the OOD restriction proved to be due to the training recipe rather than to a design flaw of BRAVE, as shown by the comparison with the IRCAM percussion baseline. The external baseline was substantially more reactive, with a forward difference of 1.75 versus 0.0045 in drums_v1 (more than two orders of magnitude greater responsiveness under this diagnostic), and produced output of approximately 0.001 when given silent input. Iterating through the training cycle repeatedly enabled the characterisation of five failure modes of RAVE-family models on small datasets: (i) posterior collapse (v1), (ii) latent underutilisation (v2), (iii) discriminator dominance or generator mode collapse (v3, v4, drums_v2), (iv) discriminator collapse (v5), and (v) OOD decoder behaviour (v6, drums_v1).

6.2 Contributions

One of the major contributions was documenting a step-by-step empirical method for uncovering RAVE-family training failures. This approach includes comparing responsive checkpoints with collapsed ones and, through this, setting concrete thresholds for verifying the model’s internal representations (§3.4). Measuring pairwise differences in both the latent vectors and the audio outputs allows for an evaluation that goes far beyond informal listening tests alone.

The second major contribution is a failure taxonomy arranged in five modes. This thesis interprets each TensorBoard signature — for example, KL drift, logit gaps between real and fake, and output amplitude degeneration — and connects it to the corresponding optimisation issue (Chapter 5). It is therefore recommended that this failure-mode classification serve as a troubleshooting guide for researchers working with RAVE training on small or heterogeneous data sets, where training failure leading to loss of stability is a frequent challenge.

A third significant contribution is that the system was benchmarked externally. The restricted reaction to OOD vocal inputs observed in this study was due to the choice of training data and hyperparameter settings rather than to an inherent weakness of the BRAVE architecture (Chapter 5). In other words, with further refinement of the training recipe, the cross-modal gap may be reduced further.

6.3 Limitations

Several limitations of this study should be considered. First, the reported latency of approximately 7 ms was estimated analytically based on configuration parameters and reported buffer sizes (§3.1). This estimate was not corroborated by calibrated loopback measurements, which would be necessary to determine the actual end-to-end delay experienced in a real-world performance environment.

Secondly, the assessment of audio quality and reactivity was done without using any formal auditory perceptual measures such as MUSHRA (Multiple Stimuli with Hidden Reference and Anchor) or MOS (Mean Opinion Score). All listening tests were informal and performed by a single author, introducing bias and affecting the generalisability of the qualitative results (Chapter 5).

Thirdly, the data sources were quite limited in size: the guitar dataset contained only about 16 hours and the drum dataset about 11 hours, which is considerably less than many community baselines for high-fidelity synthesis (§3.3).

Fourthly, the STFT magnitude correlation criterion proposed in §3.4 turned out to be insufficient as a discriminator between working and collapsed checkpoints (see §5.1.4). Near-silent outputs from collapsed models produced correlations comparable to or higher than the working `guitar_v6` canonical checkpoint, because the metric captures shared general spectral shape rather than genuine reconstruction. Output amplitude and informal listening evaluation were therefore retained as the operational evaluators; future work should re-derive the reconstruction-validation threshold against both working and collapsed reference models in the target setup.

Lastly, the voice evaluation was done using only one voice, hence it is still unknown whether the model is capable of generalising to different speakers or different vocal characteristics.

6.4 Future Work

Future research should involve calibrated loopback latency measurements. This testing will replace the current analytical estimates with empirical data, providing a definite confirmation of BRAVE’s performance within the Pure Data environment and checking that the latency lies within the bounds discussed by Wessel and Wright (2002) for intimate musical interaction.

Besides, a proper perceptual evaluation (e.g. MUSHRA or MOS) should be carried out to assess the in-distribution reconstruction quality of `guitar_v6`. This would enable a statistically valid evaluation of the model’s sound quality against original recordings and other real-time synthesis methods.

In the pursuit of better quality cross-domain resynthesis, the RAVE v3 architecture should be investigated. This involves Snake activations, a Descript-style discriminator, and Adaptive Instance Normalisation (AdaIN) for explicit style transfer. However, this kind of training requires hardware with at least 32 GB of VRAM, whereas only 16 GB were used in this project (Chapter 4).

Latent alignment could be tackled if paired or mixed-domain training data including both voice and instrument signals are used. It is hypothesised that exposing the model to both domains simultaneously should not only reduce the OOD distance but also foster the emergence of more responsive timbre transfer behaviour.

Equally important, there is significant scope in the creation of hybrid pipelines. Such systems could combine symbolic onset detection and classification with neural timbre transfer. The AVP dataset (Delgado et al., 2019), which contains annotated vocal percussion, could be used to develop a model capable of mapping vocal phonemes to drum events more reliably than a purely unsupervised autoencoder.

Finally, the validity of these findings should be confirmed on a larger, more homogeneous corpus. Such a study will reveal whether IRCAM-style responsiveness is attributable solely to the magnitude and uni-

formity of the data or whether additional architectural enhancements are necessary to minimise the cross-modal gap between human vocalisations and trained instrument domains.

6.5 Closing Reflection

The original goal of this project was to build a voice-to-guitar working prototype. Halfway through, however, it was redirected toward a documented case-study and failure-analysis project. This shift occurred when it became apparent that, although recognisable in-distribution reconstruction was achievable, expressive OOD control via the human voice remained elusive under the tested training conditions. The final effort was geared towards providing a reusable diagnostic framework and a systematic failure taxonomy rather than an unverifiable demonstration. The framework established here, it is suggested, provides a more robust foundation for future real-time neural synthesis research than an isolated successful demo could afford.

References

- ACIDS-IRCAM. (2024). Pretrained RAVE models [Accessed May 2026].
- Bowman, S. R., Vilnis, L., Vinyals, O., Dai, A. M., Jozefowicz, R., & Bengio, S. (2016). Generating sentences from a continuous space. *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL)*, 10–21. <https://doi.org/10.18653/v1/K16-1002>
- Caillon, A., & Esling, P. (2021). RAVE: A variational autoencoder for fast and high-quality neural audio synthesis. *arXiv preprint arXiv:2111.05011*.
- Caspe, F., Shier, J., Sandler, M., Saitis, C., & McPherson, A. (2025). Designing neural synthesizers for low-latency interaction. *arXiv preprint arXiv:2503.11562*.
- Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y., & Défossez, A. (2023). Simple and controllable music generation. *Advances in Neural Information Processing Systems*, 36.
- Cruz-Roldán, F., Amo-López, P., Maldonado-Bascón, S., & Lawson, S. S. (2002). An efficient and simple method for designing prototype filters for cosine-modulated pseudo-QMF banks. *IEEE Signal Processing Letters*, 9(1), 29–31.
- Delgado, A., McDonald, S., Xu, N., & Sandler, M. (2019). A new dataset for amateur vocal percussion analysis. *Proceedings of the 14th International Audio Mostly Conference: A Journey in Sound*, 17–22. <https://doi.org/10.1145/3356590.3356844>
- Engel, J., Hantrakul, L., Gu, C., & Roberts, A. (2020). DDSP: Differentiable digital signal processing. *Proceedings of the 8th International Conference on Learning Representations (ICLR)*.
- Engel, J., Resnick, C., Roberts, A., Dieleman, S., Norouzi, M., Eck, D., & Simonyan, K. (2017). Neural audio synthesis of musical notes with WaveNet autoencoders. *Proceedings of the 34th International Conference on Machine Learning (ICML)*.
- Evans, Z., Carr, C. J., Taylor, J., Hawley, S. H., & Pons, J. (2024). Stable audio: Fast timing-conditioned latent audio diffusion. *arXiv preprint arXiv:2402.04825*.
- Gillick, J., Roberts, A., Engel, J., Eck, D., & Bamman, D. (2019). Learning to groove with inverse sequence transformations. *Proceedings of the 36th International Conference on Machine Learning (ICML)*.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27.
- Hernández-Leo, D., Moreno, V., & Camps, I. (2012). *Guia docent pel seguiment i avaluació dels treballs fi de grau* (tech. rep.). Unitat de Suport a la Qualitat i a la Innovació Docent, Escola Superior Politècnica, Universitat Pompeu Fabra. Barcelona.
- Huang, S., Li, Q., Anil, C., Bao, X., Oore, S., & Grosse, R. B. (2018). TimbreTron: A WaveNet(CycleGAN(CQT(Audio))) pipeline for musical timbre transfer. *arXiv preprint arXiv:1811.09620*.
- Kehling, C., Abeßer, J., Dittmar, C., & Schuller, G. (2014). Automatic tablature transcription of electric guitar recordings by estimation of score- and instrument-related parameters. *Proceedings of the 17th International Conference on Digital Audio Effects (DAFx)*.
- Kingma, D. P., & Welling, M. (2013). Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*.
- Kong, J., Kim, J., & Bae, J. (2020). HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis. *Advances in Neural Information Processing Systems*, 33, 17022–17033.
- Liu, H., Chen, Z., Yuan, Y., Mei, X., Liu, X., Mandic, D., Wang, W., & Plumbley, M. D. (2023). AudioLDM: Text-to-audio generation with latent diffusion models. *Proceedings of the 40th International Conference on Machine Learning (ICML)*.
- Miyato, T., Kataoka, T., Koyama, M., & Yoshida, Y. (2018). Spectral normalization for generative adversarial networks. *Proceedings of the 6th International Conference on Learning Representations (ICLR)*.

- Pedroza, H., Abreu, W., Corey, R. M., & Roman, I. R. (2024). Guitar-TECHS: An electric guitar dataset covering techniques, musical excerpts, chords and scales using a diverse array of hardware. *Proceedings of the 27th International Conference on Digital Audio Effects (DAFx)*.
- Qian, K., Zhang, Y., Chang, S., Yang, X., & Hasegawa-Johnson, M. (2019). AutoVC: Zero-shot voice style transfer with only autoencoder loss. *Proceedings of the 36th International Conference on Machine Learning (ICML)*.
- Stowell, D., & Plumbley, M. D. (2008). *Characteristics of the beatboxing vocal style* (tech. rep. No. C4DM-TR-08-01). Centre for Digital Music, Queen Mary University of London.
- van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A., & Kavukcuoglu, K. (2016). WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*.
- Wessel, D., & Wright, M. (2002). Problems and prospects for intimate musical control of computers. *Computer Music Journal*, 26(3), 11–22. <https://doi.org/10.1162/014892602320582945>
- Xi, Q., Bittner, R. M., Pauwels, J., Ye, X., & Bello, J. P. (2018). GuitarSet: A dataset for guitar transcription. *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*.
- Zeghidour, N., Luebs, A., Omran, A., Skoglund, J., & Tagliasacchi, M. (2022). SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30, 495–507.
- Ziv, A., Kreuk, F., Gat, I., Copet, J., Roberts, A., Liscombe, J., & Adi, Y. (2024). Masked audio generation using a single non-autoregressive transformer. *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

A. Demo Setup

This appendix documents the configuration steps required to reproduce the real-time demonstration pipeline described in Chapter 4.

A.1 Required components

- Pure Data 0.55 or later.
- ACIDS-IRCAM `nn~` external (v1.6.0 used in this project).
- TorchScript model files: `guitar_v6_best.ts` (canonical guitar autoencoder) and `ircam_percussion.ts` (external baseline). Models are not committed to the source tree because of size; their generation procedures are documented in §4.3 of the main text.

A.2 Audio settings

The Pure Data audio engine must be configured at exactly 44,100 Hz (`Media` → `Audio settings`) to match the model’s training sample rate. A default rate of 48,000 Hz produces silent or unusable output. Recommended block size is 64 samples; the WASAPI driver buffer should be set to the lowest stable value (typically 10–20 ms) for live performance.

A.3 Patch reference

The patches `examples/test_guitar_v6.pd`, `examples/test_drums.pd`, and `examples/test_ircam_percussion.pd` ship with the project repository and follow the linear signal chain

$$\text{adc} \sim 1 \rightarrow \text{nn} \sim \langle \text{model} \rangle . \text{ts} \text{ forward} \rightarrow * \sim \langle \text{gain} \rangle \rightarrow \text{dac} \sim$$

with model-specific gain multipliers ($\times 20$ – $\times 50$) tuned empirically against output amplitude.

B. Audio Samples

Audio samples used and generated during the project are listed below. Sample files are not included in the PDF; they are distributed alongside the public release of the project Git repository (see §3.2 on reproducibility).

B.1 Input samples

- In-distribution guitar excerpts: five-second clips drawn from GuitarSet (Xi et al., 2018) solo-microphone files, used for reconstruction validation in §5.1.3 and §5.1.4.
- Voice samples: short author-recorded vocal phrases (sustained vowels, consonants and beatbox-style transients) used for informal OOD listening evaluation in §5.1.3, §5.2 and §5.3.

B.2 Generated samples

- `guitar_v6` reconstruction of GuitarSet excerpts: input/output spectrograms shown in Figure 5.3; audio files included in the repository.
- IRCAM percussion baseline outputs: voice-driven percussive outputs informally evaluated in §5.3.
- Synthetic drum samples used for the Path A hybrid-pipeline preliminary exploration (kick, snare, hi-hat one-shots), generated procedurally by `scripts/vpt_make_samples.py`.

C. Configurations and Compatibility Patches

C.1 Iteration configuration files

The seven .gin configuration files used across the training iterations are listed in Table 4.2 of Chapter 4. The full text of each configuration is reproduced in the project repository under configs/.

C.2 Compatibility patches applied to acids-rave 2.3.1

Four patches were required to install and run acids-rave 2.3.1 on the project environment (Python 3.14, PyTorch 2.11, scipy ≥ 1.14 , pytorch-lightning 2.6).

Patch 1 — pqmf.py line 10: scipy.signal.kaiser relocated.

```
# Before
from scipy.signal import firwin, kaiser, kaiser_beta, kaiserord
# After (scipy >= 1.14)
from scipy.signal import firwin, kaiser_beta, kaiserord
from scipy.signal.windows import kaiser
```

Patch 2 — pqmf.py line 67: kaiserord argument type.

```
# Before
N_, beta = kaiserord(atten, wc / np.pi)
# After (numpy 2.x compatibility)
N_, beta = kaiserord(atten, float(np.asarray(wc).ravel()[0]) / np.pi)
```

Patch 3 — pqmf.py line 70: firwin nyq parameter.

```
# Before
h = firwin(N, wc, window=('kaiser', beta), scale=False, nyq=np.pi)
# After (scipy >= 1.14)
h = firwin(N, wc, window=('kaiser', beta), scale=False, fs=2 * np.pi)
```

Patch 4 — model.py line 445: PyTorch Lightning 2.x API rename.

```
# Before
def validation_epoch_end(self, out):
# After (pytorch-lightning 2.0+)
def on_validation_epoch_end(self, out=[]):
```

These four patches total fewer than ten lines of changed code. Their effect is verifiable from the diffs in the project repository.

D. Project Planning

D.1 Project Charter

The full Project Charter is reproduced in the project repository under `docs/project_charter.pdf`. Its main elements are summarised in this section.

Project identification. Title: *BRAVE-Based Real-Time Voice-to-Instrument Resynthesis*. Programme: Grau en Enginyeria de Sistemes Audiovisuals, Universitat Pompeu Fabra, Escola Superior Politècnica. Author: Jofre Geli de Fuenmayor. Director: Lonce Wyse & Xavier Lizarraga. Period: December 2025 – June 2026 (active iteration cycle: 24 March – 12 June 2026; see *Project history and scope reset* below). ECTS: 20 (approximately 500 hours of student work).

General objective. To design, train, deploy and evaluate a streaming neural resynthesis pipeline that maps the human voice to instrument timbres in real time under a sub-10 ms latency budget, and to document the training instabilities and failure modes that emerge when this task is attempted on limited-scale, heterogeneous public datasets.

Specific objectives. See §1.3.

Scope. In scope: monophonic audio resynthesis; instrument-domain autoencoder training; real-time deployment via Pure Data + `nn~`; quantitative and informal qualitative evaluation; guitar and drums as case-study instruments. Out of scope: polyphonic modelling; offline batch processing; linguistic speech conversion; formal perceptual evaluation with controlled listener panels.

Deliverables. Project Charter and Gantt (this appendix); trained autoencoder checkpoints (TorchScript); diagnostic Python scripts; Pure Data deployment patches; failure taxonomy (§5.4); final memoir (this document); defence presentation; public Git repository.

Stakeholders. Author (implementation, evaluation, documentation); director (methodological guidance and milestone review); tribunal (final evaluation); future practitioners (secondary beneficiaries of the diagnostic methodology).

Constraints. Single workstation with one NVIDIA RTX 5080 GPU (16 GB VRAM); approximately 12 weeks of project duration with each training run requiring 90–120 wall-clock hours; publicly available datasets only (~16 h guitar, ~11 h drums); voice evaluation limited to the author’s voice.

Risks and mitigations. Training instabilities preventing a working autoencoder (mitigated by systematic diagnostics at each iteration and a pivot to documenting failure as contribution); insufficient dataset size (mitigated by multi-source combination and explicit noise-floor analysis); sub-10 ms latency unachievable on consumer hardware (mitigated by an analytical latency budget verified before deployment); voice remaining OOD (treated as honest finding and documented via the IRCAM external baseline); single-GPU bottleneck (planned conservatively, prefer community-validated configurations); live demonstration not perceptually convincing (reframed as case studies + failure analysis, §6.5).

D.2 Work Breakdown Structure

1. Project Initiation (Initial phase)
 - (a) Project Charter and planning document
 - (b) Literature review (RAVE, BRAVE, DDSF, WaveNet)
 - (c) Environment setup (Python, PyTorch, acids-rave compatibility patches)
 - (d) Dataset acquisition (GuitarSet, Guitar-TECHS, Groove MIDI)
 - (e) Pure Data + nn~ verification
2. Methodology Design (Initial phase)
 - (a) Architecture selection (BRAVE rationale)
 - (b) Diagnostic protocol design
 - (c) Training pipeline construction (.gin configs, .bat scripts)
 - (d) Preprocessing pipeline (mono + 44.1 kHz + LMDB build)
3. Guitar Case Study — Iterative Training (Execution phase)
 - (a) guitar_v1 – baseline c128_r10.gin → posterior collapse
 - (b) guitar_v2 – β warmup → latent underutilisation
 - (c) guitar_v3 – LATENT_SIZE 128 → 16 → partial response
 - (d) guitar_v4 – mic-only data → adversarial mode collapse
 - (e) guitar_v5 – halved discriminator → discriminator collapse
 - (f) guitar_v6 – official v2.gin + causal → partial in-distribution reconstruction
4. Drums Case Study — Iterative Training (Execution phase)
 - (a) drums_v1 – c16_r10 → partial transient response
 - (b) drums_v2 – v2.gin + causal → terminated before completion
5. External Baseline and Validation Studies (Execution phase)
 - (a) IRCAM percussion pretrained model evaluation
 - (b) STFT-magnitude-correlation cross-checkpoint calibration
 - (c) Noise-floor heterogeneity analysis
 - (d) Path A hybrid pipeline preliminary exploration
6. Documentation (Final phase)
 - (a) Chapter drafting (Chapters 1–6)
 - (b) Figure generation (TensorBoard plots, spectrograms, noise floor)
 - (c) Reference compilation and APA formatting
 - (d) Appendix-A (Demo Setup), Appendix-B (Audio Samples), Appendix-C (Configurations and Compatibility Patches), Appendix-D (Project Planning)
7. Submission and Defence (Final phase)
 - (a) LaTeX migration from Word/Google Docs draft
 - (b) Final memoir review and proofreading
 - (c) Public repository release (post-deadline)
 - (d) Defence presentation preparation and live demonstration setup

D.3 Project history and scope reset

Engagement with the thesis topic began on 3 December 2025 with initial scoping notes and literature reading on neural audio synthesis architectures (RAVE, DDSP, WaveNet, MusicGen). The BRAVE-based approach reported in this thesis was selected in early February 2026 and a first implementation attempt was made shortly afterwards. That attempt encountered persistent environment-compatibility problems — chiefly the breaking changes in `scipy` ≥ 1.14 and `pytorch-lightning` 2.x against `acids-rave` 2.3.1 documented in Chapter 4 (Patches 1–4) — which made the original toolchain non-functional. A deliberate decision was taken on 24 March 2026 to reset: the environment was rebuilt from scratch, the four `acids-rave` compatibility patches were derived, and the iteration sequence reported in this thesis was begun under the rebuilt pipeline.

The phase schedule below therefore documents the *active iteration cycle* running from 24 March 2026 to submission. Total elapsed engagement with the topic, including the December–March scoping and environment-rebuild work, spans roughly six months. The compressed twelve-week active cycle is the direct consequence of the restart, not of late initiation, and is also the reason the four `acids-rave` compatibility patches appear as one of the project’s deliverables (§4.1) rather than as a tacit prerequisite.

D.4 Phase schedule

Table D.1: Three-phase schedule of the active iteration cycle (24 March – 12 June 2026), following the UPF TFG guide (Hernández-Leo et al., 2012). December 2025 – 23 March 2026 was an earlier scoping and first (unsuccessful) environment-setup phase, described above. Phase boundaries correspond to the dated training runs in the project repository.

Phase	Duration	Effort (% ECTS)	Dates (2026)
Initial (planning + setup + baseline run)	2 weeks	~15 %	24 March – 5 April
Execution (iterative training + analysis)	7 weeks	~55 %	6 April – 21 May
Final (writing + defence preparation)	3 weeks	~30 %	22 May – 12 June